

DATA 260 – HW 4

7- Community Sports League Fixtures

SJSU ID : 020053319 | SRI PRAJEET VILLYAL

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b (run locally through Ollama)

GitHub repository: <https://github.com/villyalsriprajeet-cmd/data260-3319>

Tagged commit hash: dc48a3842ca4adbae94b2d2949901dbc4d44e1ae

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7

Part 1 - React Client

I added a React client in code/frontend, next to the existing FastAPI service in code/api/. The client talks to the backend on port 8619. Every request uses axios with withCredentials enabled, so the browser stores and sends the HttpOnly session cookie while the JavaScript code never reads it.

App, routing, hooks and props

App.jsx stores the login state and the fixture list with useState, and uses useEffect to load the fixtures. The routes are /(Home), /create, /update/:id and /delete/:id, and the CRUD pages are shown only after login. App passes onAdd, onUpdate and onDelete to the Create, Update and Delete pages as props, and each one redirects to the home page when done. main.jsx sets up React Router, and fixturesApi.js makes all calls to the backend.

useState and useEffect

```
export default function App() {
  const navigate = useNavigate();
  const [loggedIn, setLoggedIn] = useState(false); // true once the backend accepts our cookie
  const [fixtures, setFixtures] = useState([]); // records shown on Home
  // Load the list; 200 means the session cookie is valid, 401 means not logged in

  useEffect(() => {
    fetchFixtures()
      .then((data) => {
        setFixtures(data);
        setLoggedIn(true);
      })
      .catch(() => {
        setFixtures([]);
        setLoggedIn(false);
      });
  }, [loggedIn]);
```

React Router routes and props

```
<Navbar loggedIn={loggedIn} />
<main className="page">
  <Login loggedIn={loggedIn} setLoggedIn={setLoggedIn} />
  <Routes>
    <Route path="/" element={<Home fixtures={fixtures} loggedIn={loggedIn} />} />
    <Route path="/create" element={<Protected loggedIn={loggedIn}><CreateRecord onAdd={onAdd} /></Protected>} />
    <Route path="/update/:id" element={<Protected loggedIn={loggedIn}><UpdateRecord onUpdate={onUpdate} /></Protected>} />
    <Route path="/delete/:id" element={<Protected loggedIn={loggedIn}><DeleteRecord onDelete={onDelete} /></Protected>} />
  </Routes>
</main>
</>
);
}
```

login guard for the CRUD routes

```
function Protected({ loggedIn, children }) {
  if (!loggedIn) {
    return (
      <section className="panel">
        <h2>Login required</h2>
        <p className="muted">Log in above to manage fixtures.</p>
      </section>
    );
  }
  return children;
}
```

BrowserRouter

```
code > frontend > src > main.jsx
1  // Entry point: mounts App inside the router
2  import React from "react";
3  import ReactDOM from "react-dom/client";
4  import { BrowserRouter } from "react-router-dom";
5  import App from "../App.jsx";
6  import "../styles.css";
7  ReactDOM.createRoot(document.getElementById("root")).render(
8    <React.StrictMode>
9      <BrowserRouter>
10       <App />
11     </BrowserRouter>
12   </React.StrictMode>
13 );
```

axios client with the session cookie

```
code > frontend > src > api > JS fixturesApi.js > updateFixture
1  // Calls to the FastAPI backend on port 8619
2  import axios from "axios";
3  const api = axios.create({
4    baseURL: "http://localhost:8619",
5    withCredentials: true, // send and receive the HttpOnly session cookie
6  });
7  export async function login(email, password) {
8    const res = await api.post("/login", { email, password });
9    return res.data;
10 }
```

Main.jsx

```
code > frontend > src > main.jsx
1  // Entry point: mounts App inside the router
2  import React from "react";
3  import ReactDOM from "react-dom/client";
4  import { BrowserRouter } from "react-router-dom";
5  import App from "../App.jsx";
6  import "../styles.css";
7  ReactDOM.createRoot(document.getElementById("root")).render(
8    <React.StrictMode>
9      <BrowserRouter>
10       <App />
11     </BrowserRouter>
12   </React.StrictMode>
13 );
14
```

App.jsx

```
code > frontend > src > App.jsx > App > useEffect() callback
1 // Main app: login state, fixture list, routes, and the CRUD handlers passed as props
2 import React, { useEffect, useState } from "react";
3 import { Routes, Route, useNavigate } from "react-router-dom";
4 import Navbar from "../components/Navbar.jsx";
5 import Login from "../components/Login.jsx";
6 import Home from "../pages/Home.jsx";
7 import CreateRecord from "../pages/CreateRecord.jsx";
8 import UpdateRecord from "../pages/UpdateRecord.jsx";
9 import DeleteRecord from "../pages/DeleteRecord.jsx";
10 import { fetchFixtures, createFixture, updateFixture, deleteFixture } from "../api/fixturesApi.js";
11 // Shows the page only when logged in
12 function Protected({ loggedIn, children }) {
13   if (!loggedIn) {
14     return (
15       <section className="panel">
16         <h2>Login required</h2>
17         <p className="muted">Log in above to manage fixtures.</p>
18       </section>
19     );
20   }
21   return children;
22 }
23 export default function App() {
24   const navigate = useNavigate();
25   const [loggedIn, setLoggedIn] = useState(false); // true once the backend accepts our cookie
26   const [fixtures, setFixtures] = useState([]); // records shown on Home
27   // Load the list; 200 means the session cookie is valid, 401 means not logged in
28   useEffect(() => {
29     fetchFixtures()
30       .then((data) => {
31         setFixtures(data);
32         setLoggedIn(true);
33       })
34       .catch(() => {
35         setFixtures([]);
36         setLoggedIn(false);
37       });
38   }, [loggedIn]);
39   // Create, passed to CreateRecord as a prop
40   async function onAdd(newFixture) {
41     const created = await createFixture(newFixture); // MySQL assigns the auto-increment id
42     setFixtures([...fixtures, created]);
43     navigate("/");
44   }
45   // Update, passed to UpdateRecord as a prop
46   async function onUpdate(id, changes) {
47     const updated = await updateFixture(id, changes);
48     setFixtures(fixtures.map((f) => (f.id === id ? updated : f)));
49     navigate("/");
50   }
51   // Delete, passed to DeleteRecord as a prop
52   async function onDelete(id) {
53     await deleteFixture(id);
54     setFixtures(fixtures.filter((f) => f.id !== id));
55     navigate("/");
56   }
57   return (
58     <
59       <Navbar loggedIn={loggedIn} />
60       <main className="page">
61         <Login loggedIn={loggedIn} setLoggedIn={setLoggedIn} />
62         <Routes>
63           <Route path="/" element={<Home fixtures={fixtures} loggedIn={loggedIn} />} />
64           <Route path="/create" element={<Protected loggedIn={loggedIn}><CreateRecord onAdd={onAdd} /></Protected>} />
65           <Route path="/update/:id" element={<Protected loggedIn={loggedIn}><UpdateRecord onUpdate={onUpdate} /></Protected>} />
66           <Route path="/delete/:id" element={<Protected loggedIn={loggedIn}><DeleteRecord onDelete={onDelete} /></Protected>} />
67         </Routes>
68       </main>
69     </>
70   );
71 }
```


fixturesApi.js

```
code > frontend > src > api > JS fixturesApi.js > ...
1  // Calls to the FastAPI backend on port 8619
2  import axios from "axios";
3  const api = axios.create({
4    |   baseURL: "http://localhost:8619",
5    |   withCredentials: true, // send and receive the HttpOnly session cookie
6  | });
7  export async function login(email, password) {
8    |   const res = await api.post("/login", { email, password });
9    |   return res.data;
10 | }
11 export async function fetchFixtures() {
12 |   const res = await api.get("/fixtures");
13 |   return res.data;
14 | }
15 export async function fetchFixtureById(id) {
16 |   const res = await api.get(`/fixtures/${id}`);
17 |   return res.data;
18 | }
19 export async function createFixture(payload) {
20 |   const res = await api.post("/fixtures", payload);
21 |   return res.data;
22 | }
23 export async function updateFixture(id, payload) {
24 |   const res = await api.put(`/fixtures/${id}`, payload);
25 |   return res.data;
26 | }
27 export async function deleteFixture(id) {
28 |   const res = await api.delete(`/fixtures/${id}`);
29 |   return res.data;
30 | }
```

Login and access control

Login.jsx sends the email and password to the backend, which creates a server side session and returns an HttpOnly cookie containing only the session token. Until the user logs in, the page shows "Login required", no records are listed, and the Add Fixture link is hidden. A wrong password shows "Invalid email or password".

login form and submit

```
async function handleSubmit(e) {
  e.preventDefault();
  try {
    await login(email, password); // the browser keeps the cookie, our code never reads it
    setError("");
    setLoggedIn(true);
  } catch {
    setError("Invalid email or password");
  }
}

if (loggedIn) {
  return <p className="status">Logged in</p>;
}

return (
  <section className="panel login-panel">
    <h2>Login required</h2>
    <form className="login-form" onSubmit={handleSubmit}>
      <input type="email" placeholder="Email" value={email} onChange={(e) => setEmail(e.target.value)} required />
      <input type="password" placeholder="Password" value={password} onChange={(e) => setPassword(e.target.value)} required />
      <button type="submit" className="btn btn-green">Login</button>
    </form>
    {error && <p className="error">{error}</p>}
  </section>
);
```

hide Add Fixture until login

```
    <header className="site-header">
      <Link to="/" className="site-title"> Community League Fixtures</Link>
      <nav className="site-nav">
        <Link to="/">Home</Link>
        {loggedIn && <Link to="/create">Add Fixture</Link>}
      </nav>
    </header>
  );
}
```

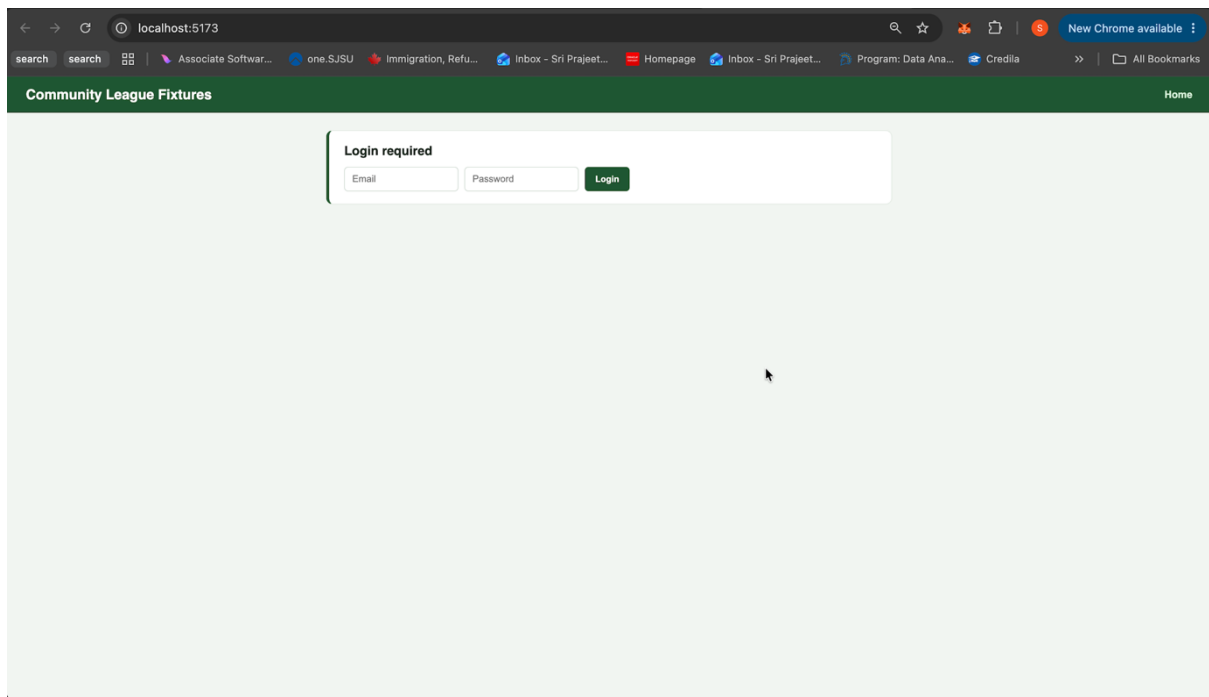
Navbar.jsx

```
code > frontend > src > components > ⚙️ Navbar.jsx > 📦 Navbar
1  // Header: league name, Home link, and Add Fixture only after login
2  import React from "react";
3  import { Link } from "react-router-dom";
4  export default function Navbar({ loggedIn }) {
5    return (
6      <header className="site-header">
7        <Link to="/" className="site-title"> Community League Fixtures</Link>
8        <nav className="site-nav">
9          <Link to="/">Home</Link>
10         {loggedIn && <Link to="/create">Add Fixture</Link>}
11       </nav>
12     </header>
13   );
14 }
```

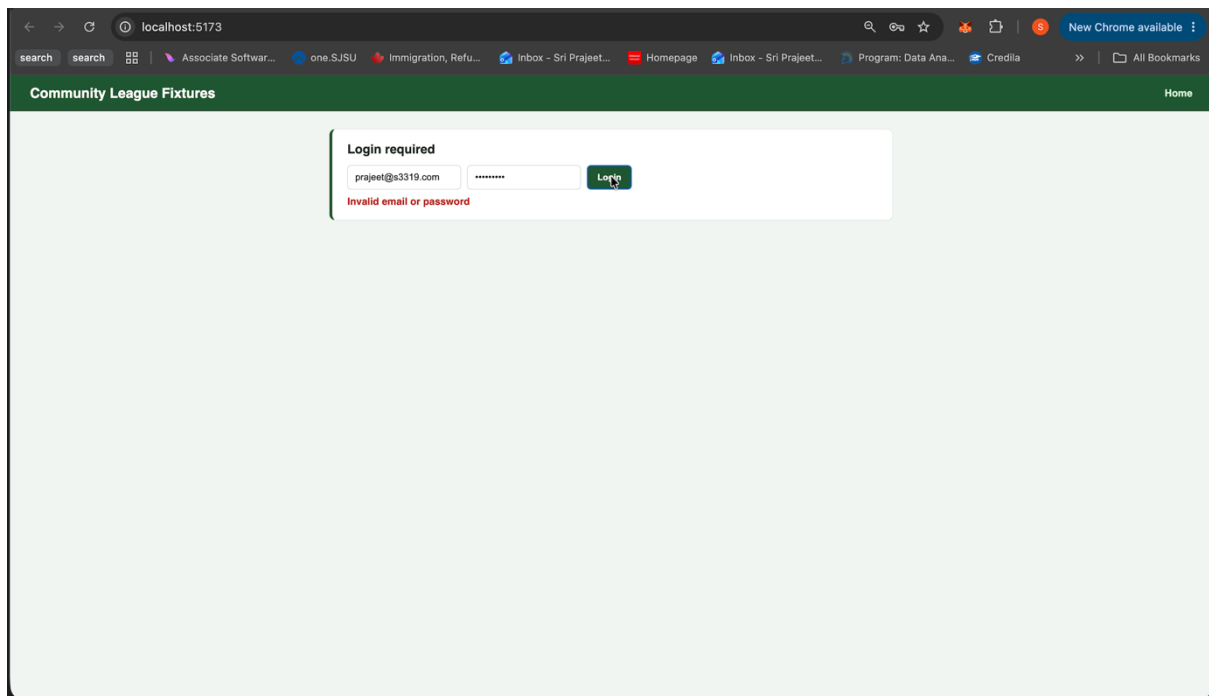
Login.jsx

```
code > frontend > src > components > Login.jsx > Login
1 // Login: email + password; the backend answers with an HttpOnly session cookie
2 import React, { useState } from "react";
3 import { login } from "../api/fixturesApi.js";
4 export default function Login({ loggedIn, setLoggedIn }) {
5   const [email, setEmail] = useState("");
6   const [password, setPassword] = useState("");
7   const [error, setError] = useState("");
8   async function handleSubmit(e) {
9     e.preventDefault();
10    try {
11      await login(email, password); // the browser keeps the cookie, our code never reads it
12      setError("");
13      setLoggedIn(true);
14    } catch {
15      setError("Invalid email or password");
16    }
17  }
18  if (loggedIn) {
19    return <p className="status">Logged in</p>;
20  }
21  return (
22    <section className="panel login-panel">
23      <h2>Login required</h2>
24      <form className="login-form" onSubmit={handleSubmit}>
25        <input type="email" placeholder="Email" value={email} onChange={(e) => setEmail(e.target.value)} required />
26        <input type="password" placeholder="Password" value={password} onChange={(e) => setPassword(e.target.value)} required />
27        <button type="submit" className="btn btn-green">Login</button>
28      </form>
29      {error && <p className="error">{error}</p>}
30    </section>
31  );
32 }
```

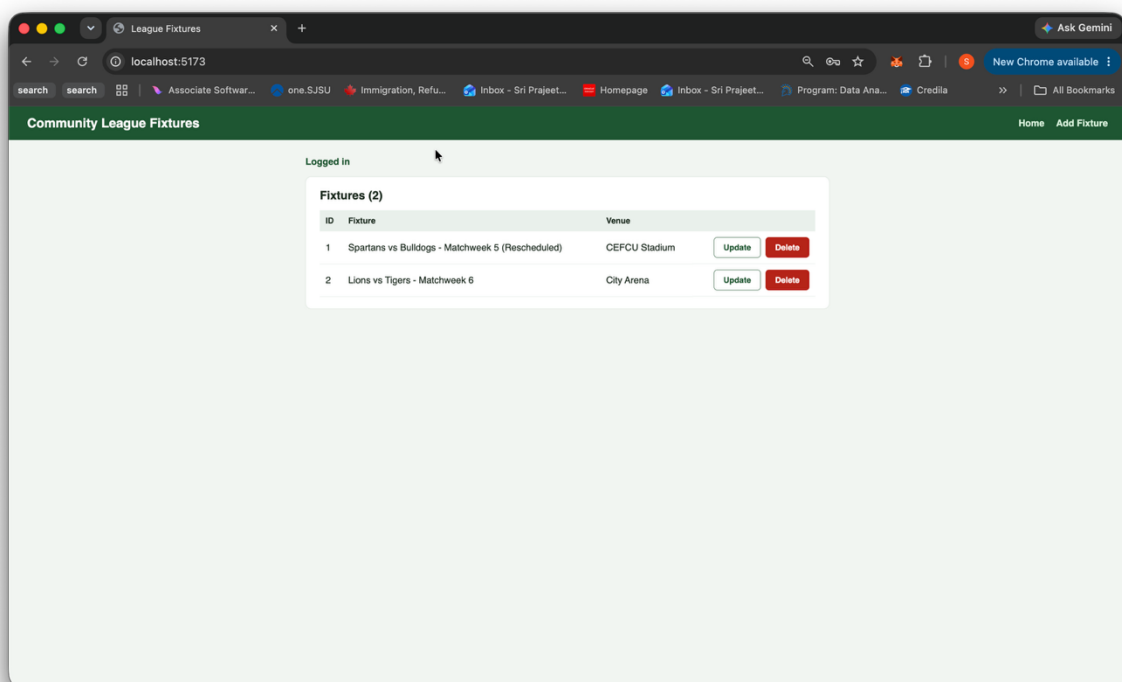
logged out: "Login required", Add Fixture hidden, no records shown



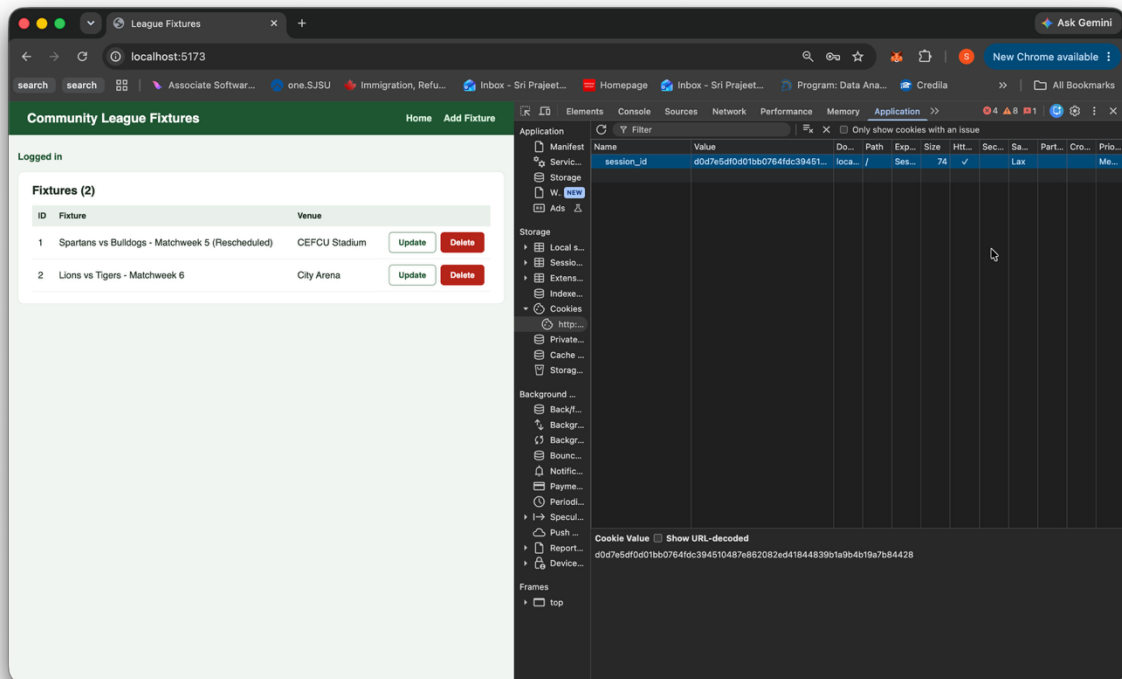
wrong password: "Invalid email or password"



After login



session_id cookie



I. Home page

Home.jsx is rendered. For a logged in user it shows every fixture from MySQL in a table, with Update and Delete buttons for each row.

```
if (!loggedIn) return null; // the Login panel above shows "Login required"
return (
  <section className="panel">
    <h2>Fixtures ({fixtures.length})</h2>
    {fixtures.length === 0 ? (
      <p className="muted">No fixtures yet.</p>
    ) : (
      <table className="fixture-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Fixture</th>
            <th>Venue</th>
            <th></th>
          </tr>
        </thead>
        <tbody>
          {fixtures.map((f) => (
            <tr key={f.id}>
              <td>{f.id}</td>
              <td>{f.fixture_title}</td>
              <td>{f.venue}</td>
              <td className="row-actions">
                <Link to={`/update/${f.id}`} className="btn btn-outline">Update</Link>
                <Link to={`/delete/${f.id}`} className="btn btn-red">Delete</Link>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </section>
)
```

Home.jsx

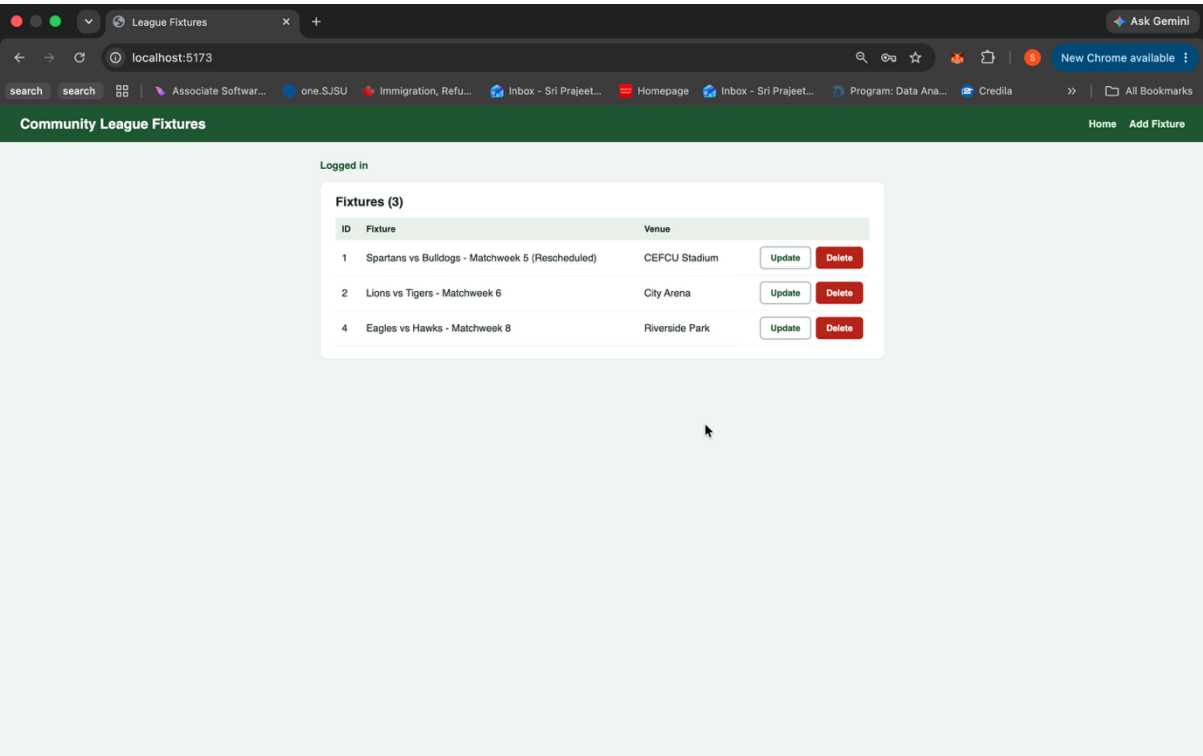
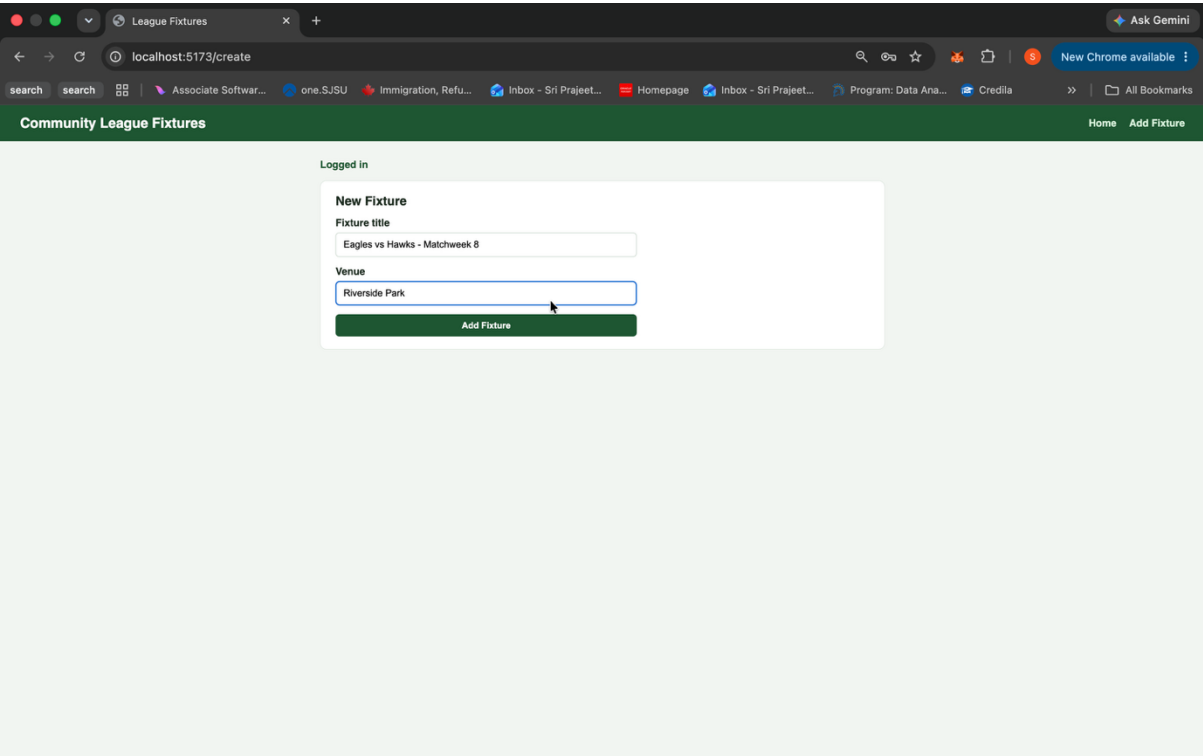
```
code > frontend > src > pages > Home.jsx > ...
1 // Home page at "/": the full fixture list for logged in users
2 import React from "react";
3 import { Link } from "react-router-dom";
4 export default function Home({ fixtures, loggedIn }) {
5   if (!loggedIn) return null; // the Login panel above shows "Login required"
6   return (
7     <section className="panel">
8       <h2>Fixtures ({fixtures.length})</h2>
9       {fixtures.length === 0 ? (
10         <p className="muted">No fixtures yet.</p>
11       ) : (
12         <table className="fixture-table">
13           <thead>
14             <tr>
15               <th>ID</th>
16               <th>Fixture</th>
17               <th>Venue</th>
18               <th></th>
19             </tr>
20           </thead>
21           <tbody>
22             {fixtures.map((f) => (
23               <tr key={f.id}>
24                 <td>{f.id}</td>
25                 <td>{f.fixture_title}</td>
26                 <td>{f.venue}</td>
27                 <td className="row-actions">
28                   <Link to={` /update/${f.id}`} className="btn btn-outline">Update</Link>
29                   <Link to={` /delete/${f.id}`} className="btn btn-red">Delete</Link>
30                 </td>
31               </tr>
32             ))}
33           </tbody>
34         </table>
35       )}
36     </section>
37   );
38 }
```

II. Add a new record

CreateRecord.jsx is shown on /create and has inputs for the fixture title and venue with an Add Fixture button. On submit, the onAdd prop saves the fixture to MySQL, which assigns an auto increment ID, and the user is redirected to the home page. The new fixture got ID 4 because MySQL does not reuse the ID of the fixture deleted.

```
code > frontend > src > pages > CreateRecord.jsx > ...
1 // Create page at "/create": title and venue inputs, saving goes through the onAdd prop
2 import React, { useState } from "react";
3 export default function CreateRecord({ onAdd }) {
4   const [fixtureTitle, setFixtureTitle] = useState("");
5   const [venue, setVenue] = useState("");
6   function handleSubmit(e) {
7     e.preventDefault();
8     onAdd({ fixture_title: fixtureTitle, venue: venue }); // parent POSTs and redirects home
9   }
10  return (
11    <section className="panel">
12      <h2>New Fixture</h2>
13      <form className="fixture-form" onSubmit={handleSubmit}>
14        <label>Fixture title<input value={fixtureTitle} onChange={(e) => setFixtureTitle(e.target.value)} required /></label>
15        <label>Venue<input value={venue} onChange={(e) => setVenue(e.target.value)} required /></label>
16        <button type="submit" className="btn btn-green">Add Fixture</button>
17      </form>
18    </section>
19  );
20 }
```

Create Form



III. Update a record

UpdateRecord.jsx is shown on /update/:id and uses useEffect to load the fixture into the inputs. On submit, the onUpdate prop saves the changes to MySQL, and the user is redirected to the home page with the updated list.

```
code > frontend > src > pages > UpdateRecord.jsx > ...
1 // Update page at "/update/:id": loads the fixture, saving goes through the onUpdate prop
2 import React, { useEffect, useState } from "react";
3 import { useParams } from "react-router-dom";
4 import { fetchFixtureById } from "../api/fixturesApi.js";
5 export default function UpdateRecord({ onUpdate }) {
6   const fixtureId = Number(useParams().id); // id from the URL
7   const [fixtureTitle, setFixtureTitle] = useState("");
8   const [venue, setVenue] = useState("");
9   // Fill the inputs with the saved values
10  useEffect(() => {
11    fetchFixtureById(fixtureId).then((f) => {
12      setFixtureTitle(f.fixture_title);
13      setVenue(f.venue);
14    });
15  }, [fixtureId]);
16  function handleSubmit(e) {
17    e.preventDefault();
18    onUpdate(fixtureId, { fixture_title: fixtureTitle, venue: venue }); // parent PUTs and redirects home
19  }
20  return (
21    <section className="panel">
22      <h2>Update Fixture #{fixtureId}</h2>
23      <form className="fixture-form" onSubmit={handleSubmit}>
24        <label>Fixture title<input value={fixtureTitle} onChange={(e) => setFixtureTitle(e.target.value)} required /></label>
25        <label>Venue<input value={venue} onChange={(e) => setVenue(e.target.value)} required /></label>
26        <button type="submit" className="btn btn-green">Update Fixture</button>
27      </form>
28    </section>
29  );
30 }
```

Update form

League Fixtures

localhost:5173/update/2

Community League Fixtures

Home Add Fixture

Logged in

Update Fixture #2

Fixture title

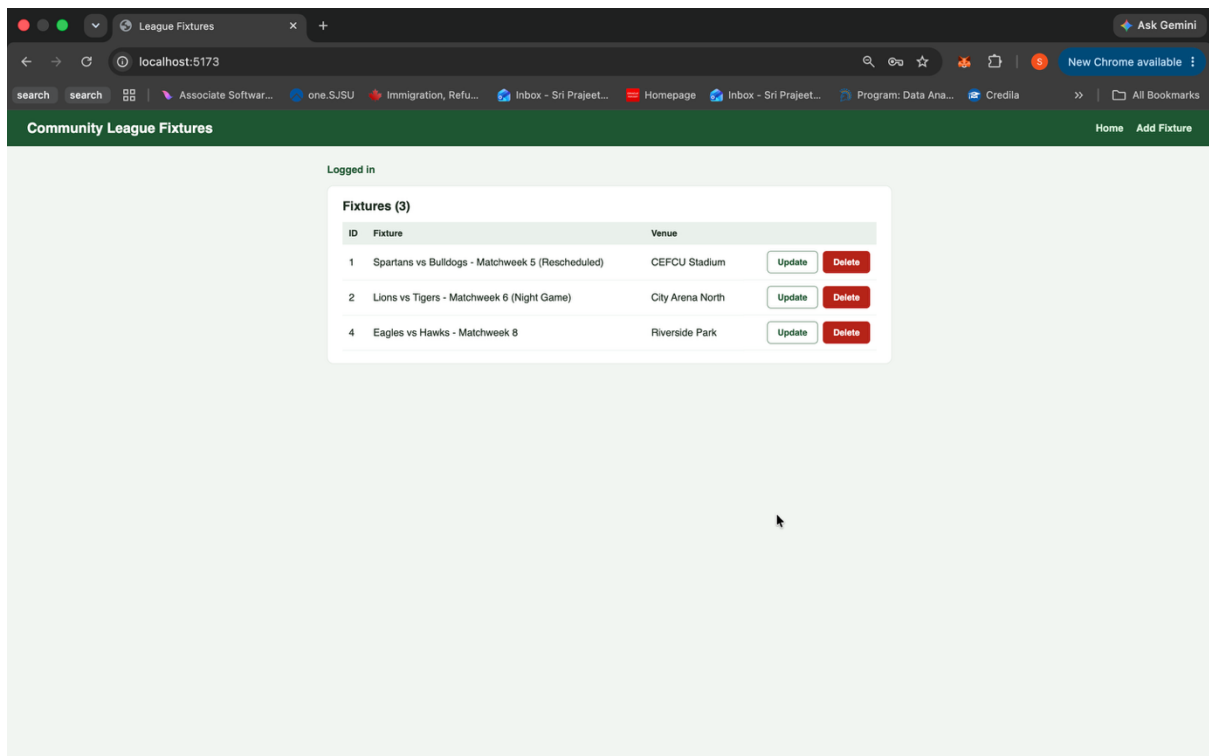
Lions vs Tigers - Matchweek 6 (Night Game)

Venue

City Arena North

Update Fixture

shows the new title and venue

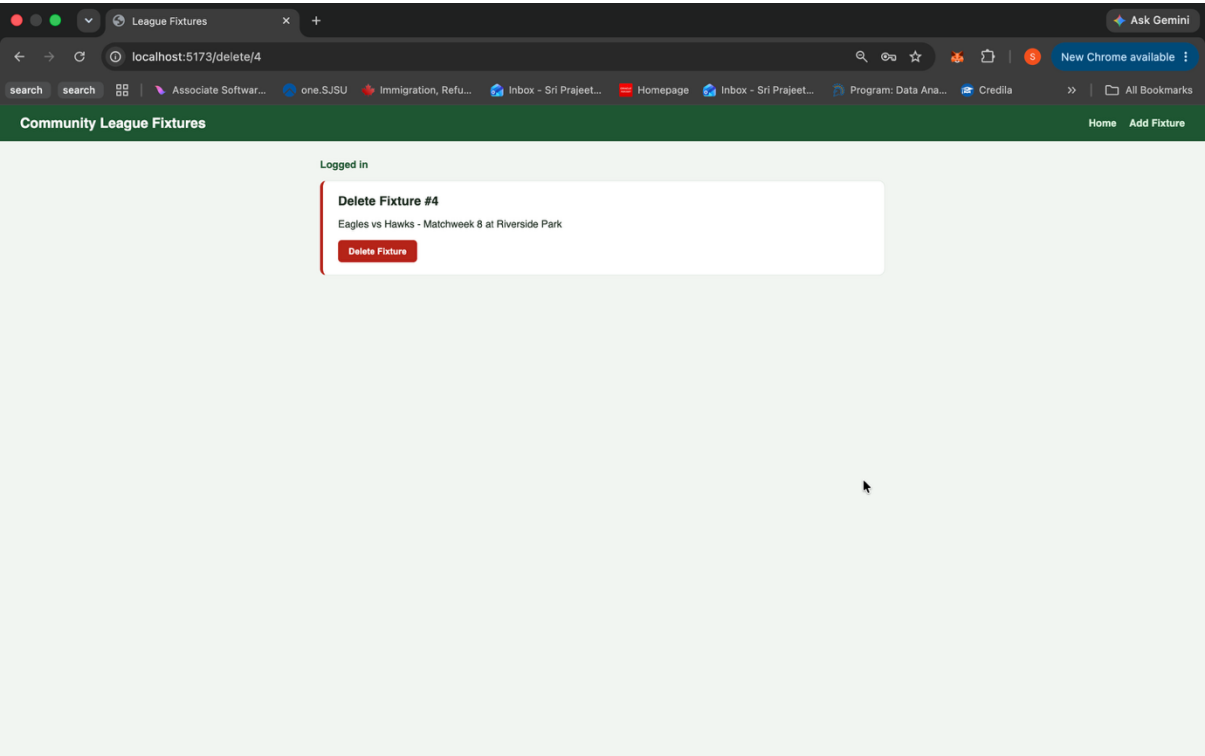


IV. Delete a record

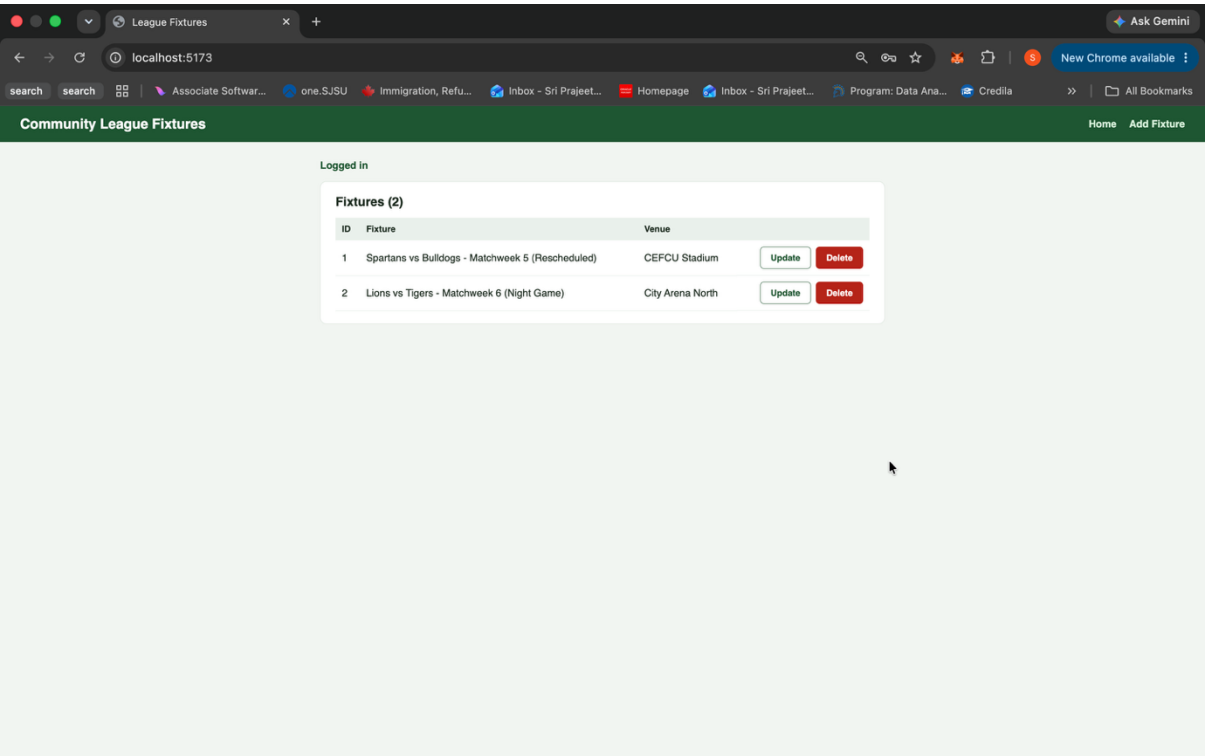
DeleteRecord.jsx is shown on `/delete/:id` and displays the selected fixture with a Delete Fixture button. Clicking it calls the `onDelete` prop, which removes the fixture from MySQL and redirects the user to the home page.

```
code > frontend > src > pages > DeleteRecord.jsx > ...
1 // Delete page at "/delete/:id": shows the fixture, deleting goes through the onDelete prop
2 import React, { useEffect, useState } from "react";
3 import { useParams } from "react-router-dom";
4 import { fetchFixtureById } from "../api/fixturesApi.js";
5 export default function DeleteRecord({ onDelete }) {
6   const fixtureId = Number(useParams().id); // id from the URL
7   const [fixture, setFixture] = useState(null);
8   // Load the fixture so the user sees what will be removed
9   useEffect(() => {
10     fetchFixtureById(fixtureId).then(setFixture).catch(() => setFixture(null));
11   }, [fixtureId]);
12   if (!fixture) {
13     return <section className="panel"><p className="muted">Fixture not found.</p></section>;
14   }
15   return (
16     <section className="panel danger-panel">
17       <h2>Delete Fixture #{fixtureId}</h2>
18       <p>{fixture.fixture_title} at {fixture.venue}</p>
19       <button className="btn btn-red" onClick={() => onDelete(fixtureId)}>Delete Fixture</button>
20     </section>
21   );
22 }
```

Delete Record



fixture 4 is removed



Changes persisted in MySQL

After the UI tests, the fixtures table shows fixture 2 updated and fixture 4 removed, confirming that the React changes were saved in MySQL.

```
● (data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; SELECT * FROM fixtures;"
Enter password:
+-----+-----+-----+
| id | fixture_title | venue | home_team_id |
+-----+-----+-----+
| 1 | Spartans vs Bulldogs - Matchweek 5 (Rescheduled) | CEFCU Stadium | NULL |
| 2 | Lions vs Tigers - Matchweek 6 (Night Game) | City Arena North | NULL |
+-----+-----+-----+
```

Part 2 - MySQL Persistence and Server Side Sessions

I extended my existing FastAPI service in code/api/. The fixture data moved from an in memory Python list to MySQL through SQLAlchemy, and the HW3 signed cookie login was replaced by server side sessions stored in a sessions table. The browser cookie now holds only an opaque random token.

Database and tables

The schema is committed as a migration file, code/api/sql/001_schema.sql. It creates the database s3319_rel with four tables: fixtures (my domain entity: auto-increment id, fixture_title as the primary field, venue as the secondary field), users (id, name, unique email, password_hash), sessions (id = session token, user_id, created_at, expires_at), and teams, which is the related table seeded in Part 3.

```
<> home.html <> login.html 001_schema.sql U X
code > api > sql > 001_schema.sql
1  -- HW4 schema for Domain 7 (Community sports league fixtures)
2  CREATE DATABASE IF NOT EXISTS s3319_rel CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3  USE s3319_rel;
4  -- login accounts, password stored only as a hash
5  CREATE TABLE IF NOT EXISTS users (
6      id INT AUTO_INCREMENT PRIMARY KEY,
7      name VARCHAR(100) NOT NULL,
8      email VARCHAR(255) NOT NULL UNIQUE,
9      password_hash VARCHAR(255) NOT NULL
10 );
11 -- server side sessions, the cookie only holds this id
12 CREATE TABLE IF NOT EXISTS sessions (
13     id VARCHAR(64) PRIMARY KEY,
14     user_id INT NOT NULL,
15     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
16     expires_at DATETIME NOT NULL,
17     CONSTRAINT fk_sessions_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
18 );
19 -- related table, filled with 200 seeded rows in Part 3
20 CREATE TABLE IF NOT EXISTS teams (
21     id INT AUTO_INCREMENT PRIMARY KEY,
22     name VARCHAR(100) NOT NULL,
23     city VARCHAR(100) NOT NULL
24 );
25 -- primary domain entity: id, primary field, secondary field
26 CREATE TABLE IF NOT EXISTS fixtures (
27     id INT AUTO_INCREMENT PRIMARY KEY,
28     fixture_title VARCHAR(200) NOT NULL,
29     venue VARCHAR(200) NOT NULL,
30     home_team_id INT NULL,
31     CONSTRAINT fk_fixtures_team FOREIGN KEY (home_team_id) REFERENCES teams(id) ON DELETE SET NULL
32 );
```

```
(data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; SHOW TABLES; DESCRIBE fixtures; DESCRIBE sessions;"
Enter password:
+-----+
| Tables_in_s3319_rel |
+-----+
| fixtures |
| sessions |
| teams   |
| users   |
+-----+

+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+
| id         | int       | NO   | PRI | NULL    | auto_increment |
| fixture_title | varchar(200) | NO   |     | NULL    |
| venue      | varchar(200) | NO   |     | NULL    |
| home_team_id | int       | YES  | MUL | NULL    |
+-----+

+-----+
| Field      | Type      | Null | Key | Default      | Extra      |
+-----+
| id         | varchar(64) | NO   | PRI | NULL         | |
| user_id    | int         | NO   | MUL | NULL         |
| created_at | datetime    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| expires_at | datetime    | NO   |     | NULL         |
+-----+
```

Database connection

database.py reads the MySQL URL from code/api/.env, creates the SQLAlchemy engine, and defines the connection variable db_session_basede26 as required. get_db() opens one session per request and always closes it.

```
<> home.html <> login.html 001_schema.sql requirements.txt U .env.example U database.py U X
code > api > database.py > ...
1 # Database engine and session factory for s3319_rel
2 import os
3 from pathlib import Path
4 from dotenv import load_dotenv
5 from sqlalchemy import create_engine
6 from sqlalchemy.orm import sessionmaker, declarative_base
7 load_dotenv(Path(__file__).resolve().parent / ".env") # read code/api/.env
8 DATABASE_URL = os.getenv("DATABASE_URL") # MySQL connection string
9 if not DATABASE_URL:
10     raise RuntimeError("DATABASE_URL missing in code/api/.env")
11 engine = create_engine(DATABASE_URL, pool_pre_ping=True) # checks the connection is alive before use
12 db_session_basede26 = sessionmaker(bind=engine, autoflush=False) # required database connection variable
13 Base = declarative_base() # parent class for all ORM models
14 def get_db():
15     db = db_session_basede26() # open one session per request
16     try:
17         yield db # hand it to the endpoint
18     finally:
19         db.close() # always give the connection back
20

• (data260) villyalsriprajeet@Mac data260-3319 % python -c "from code.api.database import engine; from sqlalchemy import text; c = engine.connect(); print(c.execute(text('SELECT DATABASE(), CURRENT_USER(), VERSION()')).fetchone()); c.close()"
('s3319_rel', 's3319_app@localhost', '8.0.30')
• (data260) villyalsriprajeet@Mac data260-3319 %
```

SQLAlchemy models and request/response schemas

models.py maps each table to a Python class. schemas.py defines the JSON the API accepts and returns: FixtureIn validates create/update bodies, FixtureOut shapes responses, and LoginIn is the login body.

```
code > api > models.py > ...
1 from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, func
2 from sqlalchemy.orm import relationship
3 from .database import Base
4 class User(Base):
5     __tablename__ = "users" # login accounts
6     id = Column(Integer, primary_key=True, autoincrement=True)
7     name = Column(String(100), nullable=False)
8     email = Column(String(255), nullable=False, unique=True)
9     password_hash = Column(String(255), nullable=False)
10 class SessionToken(Base):
11     __tablename__ = "sessions" # server side sessions
12     id = Column(String(64), primary_key=True) # opaque token that goes in the cookie
13     user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
14     created_at = Column(DateTime, nullable=False, server_default=func.now())
15     expires_at = Column(DateTime, nullable=False)
16 class Team(Base):
17     __tablename__ = "teams"
18     id = Column(Integer, primary_key=True, autoincrement=True)
19     name = Column(String(100), nullable=False)
20     city = Column(String(100), nullable=False)
21     fixtures = relationship("Fixture", back_populates="home_team") # all fixtures of this team
22 class Fixture(Base):
23     __tablename__ = "fixtures" # primary domain entity
24     id = Column(Integer, primary_key=True, autoincrement=True)
25     fixture_title = Column(String(200), nullable=False) # primary field
26     venue = Column(String(200), nullable=False) # secondary field
27     home_team_id = Column(Integer, ForeignKey("teams.id", ondelete="SET NULL"), nullable=True)
28     home_team = relationship("Team", back_populates="fixtures")
29
```

```
in.html 001_schema.sql requirements.txt .env.example database.py mod
code > api > schemas.py > ...
1 # Pydantic shapes for request bodies and responses
2 from pydantic import BaseModel, ConfigDict, EmailStr, Field
3 class FixtureIn(BaseModel):
4     fixture_title: str = Field(min_length=1, max_length=200) # body for create and update
5     venue: str = Field(min_length=1, max_length=200)
6 class FixtureOut(BaseModel):
7     model_config = ConfigDict(from_attributes=True) # lets it read ORM objects
8     id: int
9     fixture_title: str
10    venue: str
11 class TeamOut(BaseModel):
12     model_config = ConfigDict(from_attributes=True)
13     id: int
14     name: str
15     city: str
16 class FixtureWithTeam(FixtureOut):
17     home_team: TeamOut | None = None # related data for the Part 3 list
18 class LoginIn(BaseModel):
19     email: EmailStr # body for login
20     password: str
21
```

```

(data260) villyalsriprajeet@Mac data260-3319 % python -c "from code.api.database import db_session_based26; from code.api.models import User, SessionToken, Team, Fixture; from code
.api import schemas; s = db_session_based26(); print({m.__tablename__: s.query(m).count() for m in (User, SessionToken, Team, Fixture)}); s.close(); print('schemas OK:', schemas.Fi
xtureIn(fixture_title='Spartans vs Bulldogs', venue='Spartan Soccer Complex'))"
{'users': 0, 'sessions': 0, 'teams': 0, 'fixtures': 0}
schemas OK: fixture_title='Spartans vs Bulldogs' venue='Spartan Soccer Complex'
(data260) villyalsriprajeet@Mac data260-3319 %
```

Login with email and password and server side sessions

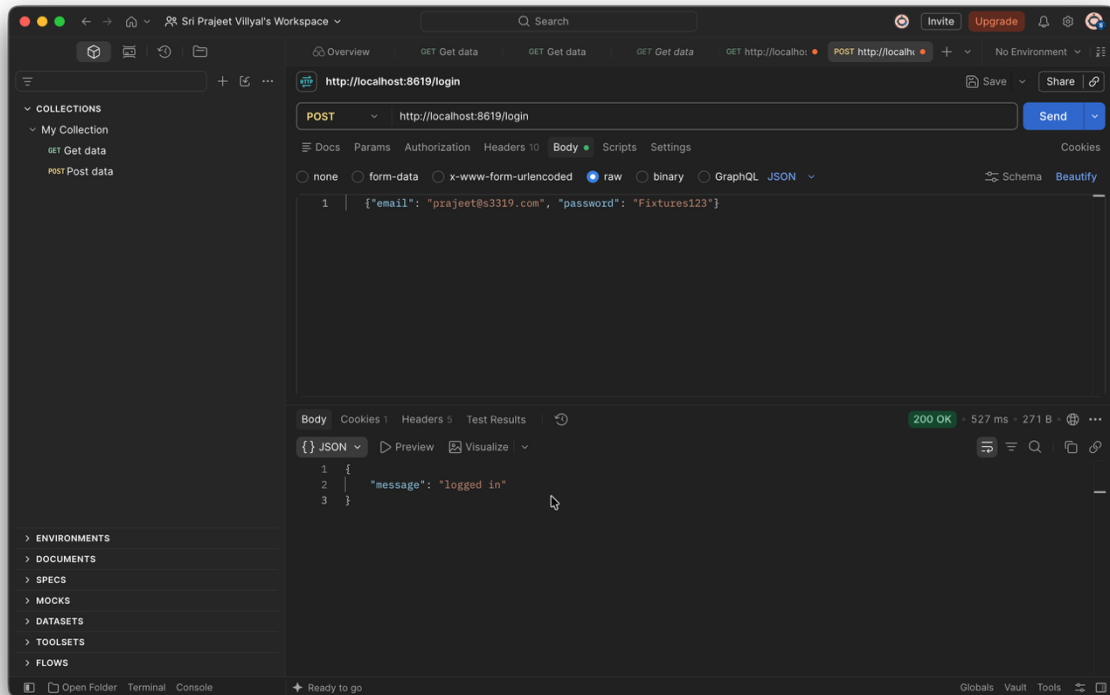
security.py hashes passwords and a random salt, so only the hash is stored in users.password_hash. On POST /login the backend checks the hash, inserts a row into sessions with a random 64 character token and an expiry time, and sets an HttpOnly cookie that contains only that token. require_session() looks the token up in the sessions table and returns 401 "Login required" if it is missing, unknown or expired.

```
code > api > security.py > create_session
1 # Password hashing and serverside session helpers
2 import hashlib
3 import hmac
4 import secrets
5 from datetime import datetime, timedelta, timezone
6 from fastapi import Depends, HTTPException, Request
7 from sqlalchemy.orm import Session
8 from .database import get_db
9 from .models import SessionToken
10 SESSION_COOKIE = "session_id" # cookie name stored in the browser
11 SESSION_TTL_MINUTES = 30 # value written to sessions.expires_at
12 def hash_password(password: str) -> str:
13     salt = secrets.token_hex(16) # random salt for this password
14     digest = hashlib.pbkdf2_hmac("sha256", password.encode(), bytes.fromhex(salt), 200_000).hex()
15     return f"{salt}${digest}" # store salt and hash together
16 def verify_password(password: str, stored: str) -> bool:
17     salt, digest = stored.split("$") # split the stored value
18     check = hashlib.pbkdf2_hmac("sha256", password.encode(), bytes.fromhex(salt), 200_000).hex()
19     return hmac.compare_digest(check, digest) # compare the two hashes
20 def utc_now() -> datetime:
21     return datetime.now(timezone.utc).replace(tzinfo=None) # UTC time for the DATETIME columns
22 def create_session(db: Session, user_id: int) -> SessionToken:
23     now = utc_now()
24     row = SessionToken(id=secrets.token_hex(32), user_id=user_id, created_at=now,
25                       expires_at=now + timedelta(minutes=SESSION_TTL_MINUTES)) # opaque 64-char token
26     db.add(row)
27     db.commit()
28     return row
29 def require_session(request: Request, db: Session = Depends(get_db)) -> SessionToken:
30     token = request.cookies.get(SESSION_COOKIE) # token from the HttpOnly cookie
31     session = db.get(SessionToken, token) if token else None # look it up in the sessions table
32     if session is None or session.expires_at < utc_now():
33         raise HTTPException(status_code=401, detail="Login required")
34     return session
```

```
code > api > routers > auth.py > ...
1 # HW4 login route: email + password, server side session, HttpOnly cookie
2 from fastapi import APIRouter, Depends, HTTPException, Response
3 from sqlalchemy.orm import Session
4 from ..database import get_db
5 from ..models import User
6 from ..schemas import LoginIn
7 from ..security import SESSION_COOKIE, create_session, verify_password
8 router = APIRouter()
9 @router.post("/login")
10 def login(payload: LoginIn, response: Response, db: Session = Depends(get_db)):
11     user = db.query(User).filter(User.email == payload.email).first() # find the account by email
12     if user is None or not verify_password(payload.password, user.password_hash):
13         raise HTTPException(status_code=401, detail="Invalid email or password")
14     session = create_session(db, user.id) # new row in the sessions table
15     response.set_cookie(key=SESSION_COOKIE, value=session.id, httponly=True, samesite="lax") # cookie holds only the token
16     return {"message": "logged in"}
```

```
(data260) villiyalsriprajeet@Mac data260-3319 % curl -i -X POST http://localhost:8619/login -H "Content-Type: application/json" -d '{"email": "prajeet@s3319.com", "password": "Fixtur
es123"}'
HTTP/1.1 200 OK
date: Thu, 24 Sep 2026 18:55:35 GMT
server: uvicorn
content-length: 23
content-type: application/json
set-cookie: session_id=c801c088d62f622b77be5803b5d4ee0300ed9139c2d7f9d1ee5f06a88215eaa7; HttpOnly; Path=/; SameSite=lax

{"message": "logged in"}
(data260) villiyalsriprajeet@Mac data260-3319 %
```



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER
• (data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; SELECT id, name, email, LEFT(password_hash, 50) AS password_hash FROM users; SELECT * FROM sessions;"
Enter password:
+----+-----+-----+-----+
| id | name | email | password_hash |
+----+-----+-----+-----+
| 1 | Prajeet | prajeet@s3319.com | 821e764aab958a951a22167029d7dfa55348496ed04a36b635 |
+----+-----+-----+-----+
+----+-----+-----+-----+
| id | user_id | created_at | expires_at |
+----+-----+-----+-----+
| c801c088d62f622b77be5803b5d4ee0300ed9139c2d7f9d1ee5f06a88215eaa7 | 1 | 2026-09-24 18:55:36 | 2026-09-24 19:25:36 |
+----+-----+-----+-----+
• (data260) villyalsriprajeet@Mac data260-3319 %

```

CRUD API for fixtures

routes/fixtures.py implements the five required operations. The router level dependency `require_session` protects every route, so only logged in users can use them.

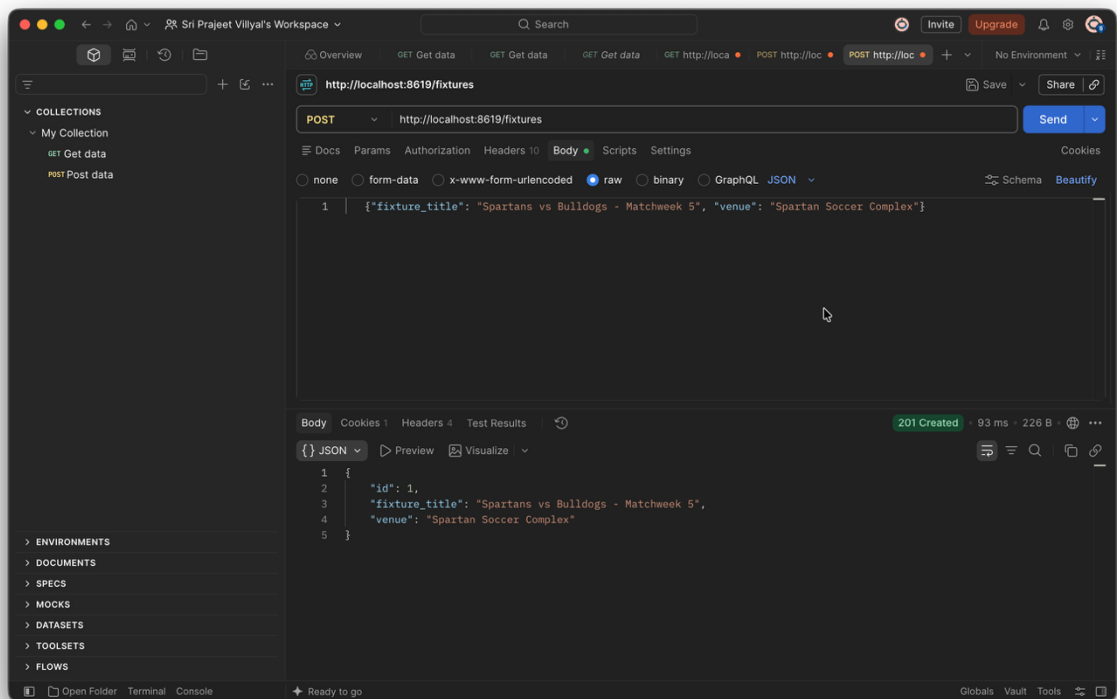
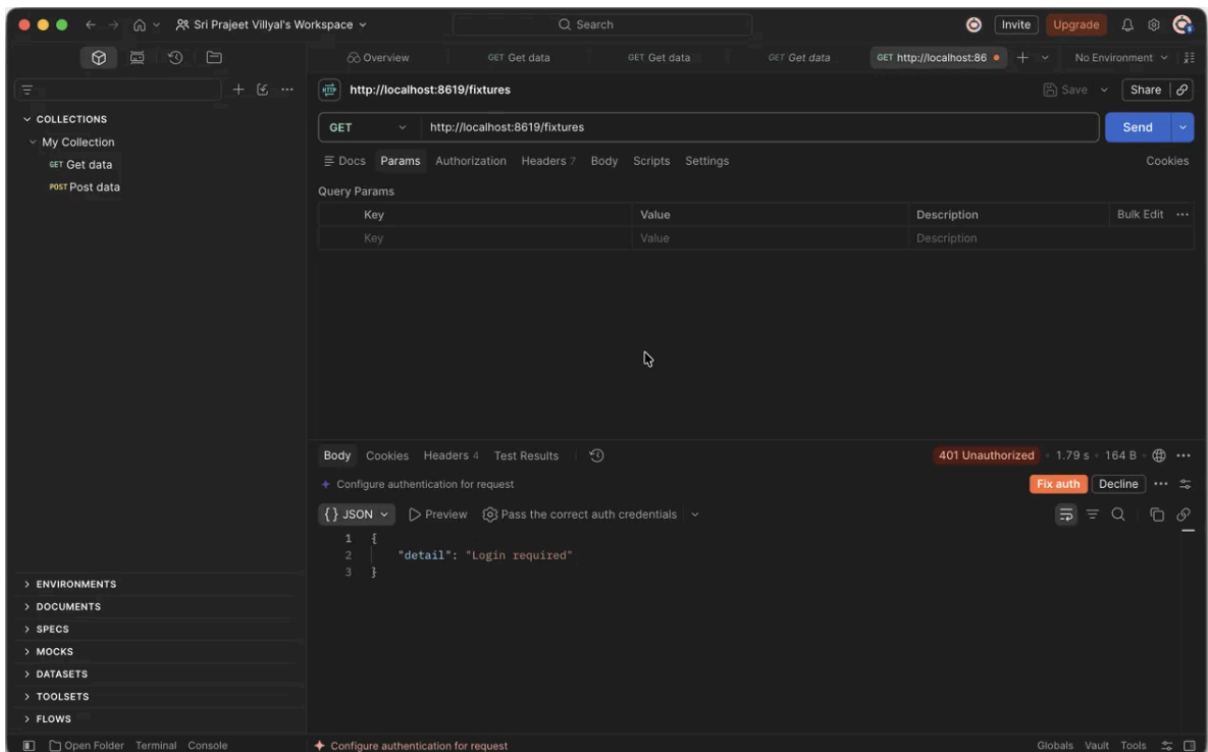
Operation	Method	Endpoint
Add a new record	POST	/fixtures
View all records	GET	/fixtures
View a record by ID	GET	/fixtures/{id}
Update record details	PUT	/fixtures/{id}
Delete a record	DELETE	/fixtures/{id}

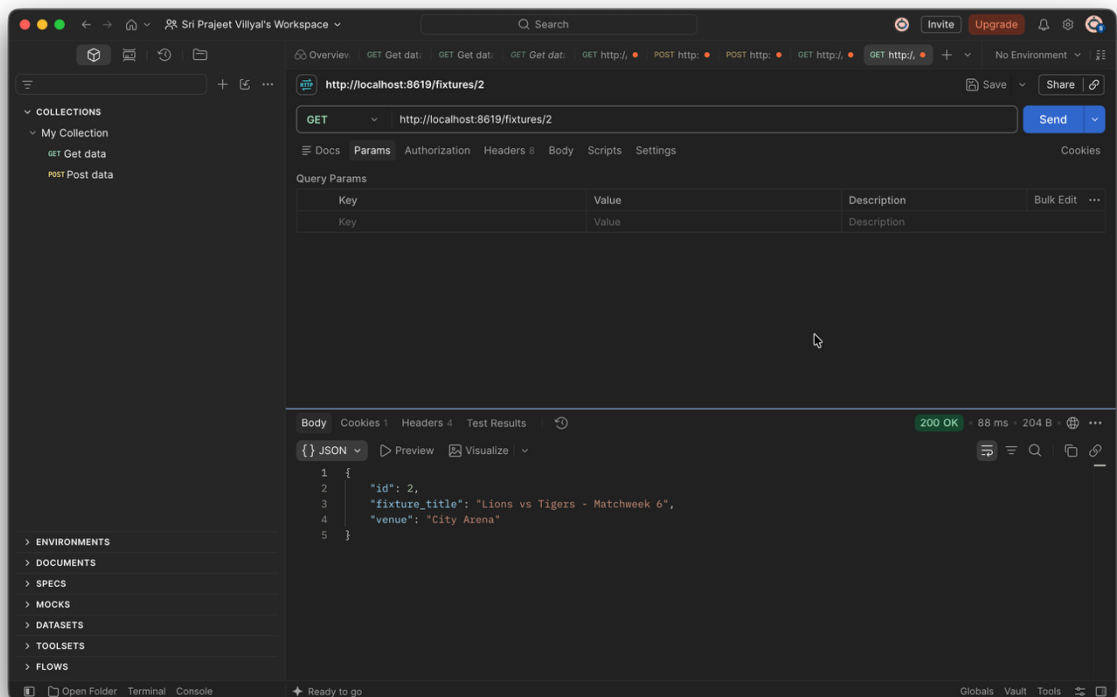
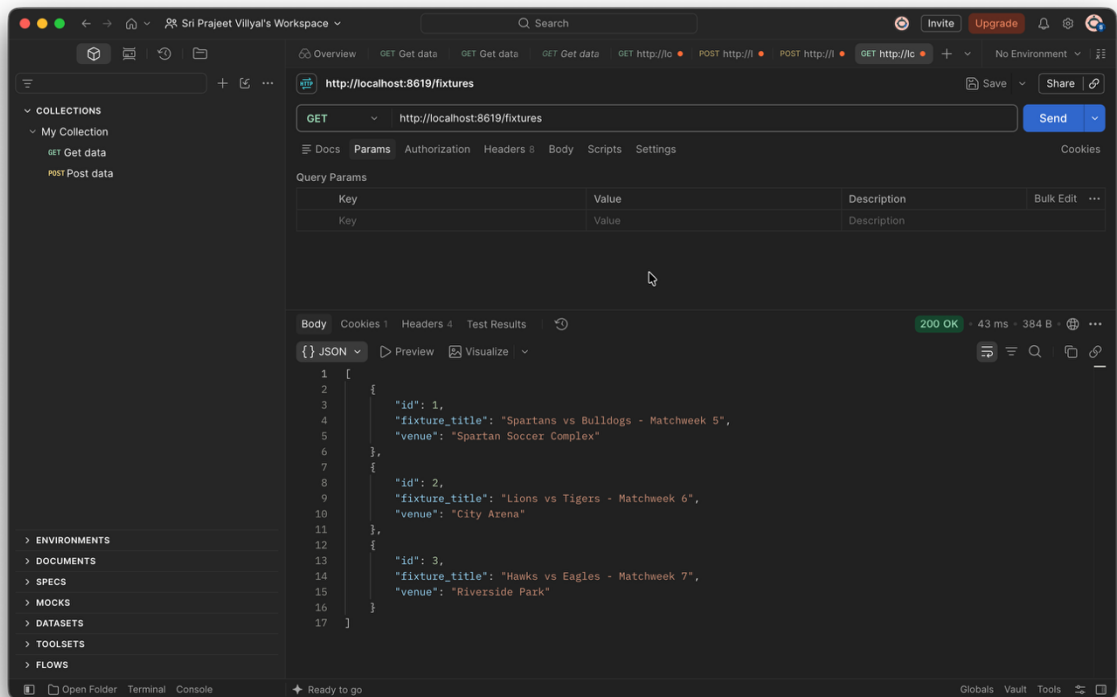
code > api > routers > fixtures.py > update_fixture

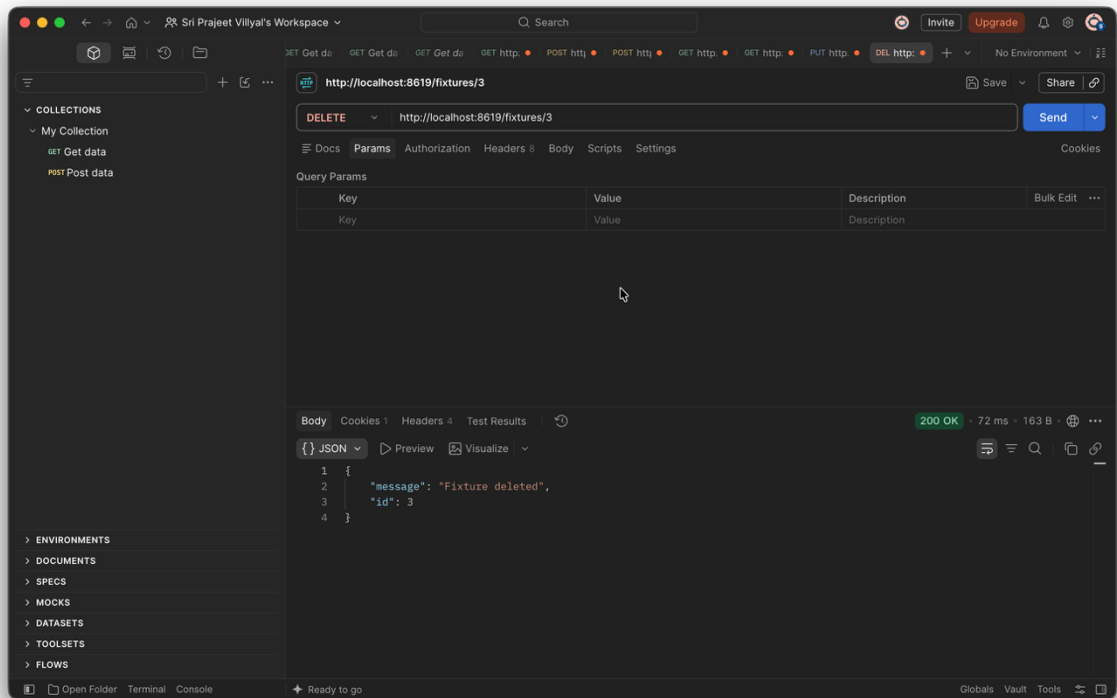
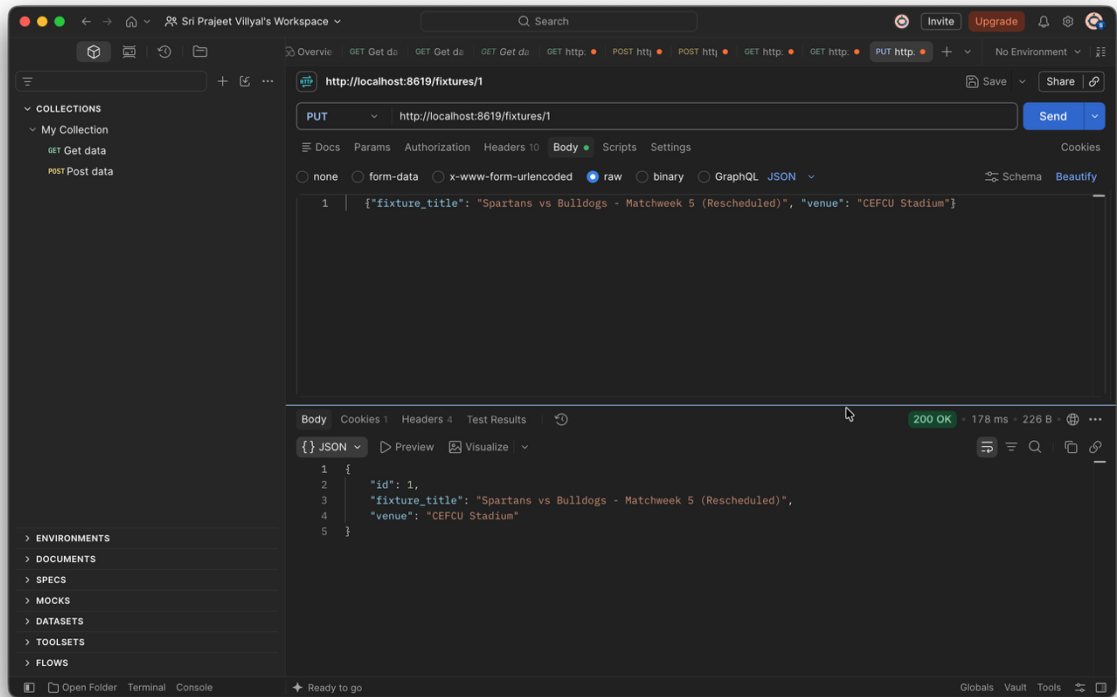
```
1 # HW4 fixture CRUD routes, every route requires a logged in session
2 from fastapi import APIRouter, Depends, HTTPException
3 from sqlalchemy.orm import Session
4 from ..database import get_db
5 from ..models import Fixture
6 from ..schemas import FixtureIn, FixtureOut
7 from ..security import require_session
8 router = APIRouter(prefix="/fixtures", dependencies=[Depends(require_session)]) # 401 unless logged in
9 @router.post("", response_model=FixtureOut, status_code=201)
10 def create_fixture(payload: FixtureIn, db: Session = Depends(get_db)):
11     fixture = Fixture(fixture_title=payload.fixture_title, venue=payload.venue) # id comes from AUTO_INCREMENT
12     db.add(fixture)
13     db.commit()
14     db.refresh(fixture) # reload to get the new id
15     return fixture
16 @router.get("", response_model=list[FixtureOut])
17 def list_fixtures(db: Session = Depends(get_db)):
18     return db.query(Fixture).order_by(Fixture.id).all() # all records
19 @router.get("/{fixture_id}", response_model=FixtureOut)
20 def get_fixture(fixture_id: int, db: Session = Depends(get_db)):
21     fixture = db.get(Fixture, fixture_id) # look up by primary key
22     if fixture is None:
23         raise HTTPException(status_code=404, detail="Fixture not found")
24     return fixture
25 @router.put("/{fixture_id}", response_model=FixtureOut)
26 def update_fixture(fixture_id: int, payload: FixtureIn, db: Session = Depends(get_db)):
27     fixture = db.get(Fixture, fixture_id)
28     if fixture is None:
29         raise HTTPException(status_code=404, detail="Fixture not found")
30     fixture.fixture_title = payload.fixture_title # new primary field
31     fixture.venue = payload.venue # new secondary field
32     db.commit()
33     db.refresh(fixture)
34     return fixture
35 @router.delete("/{fixture_id}")
36 def delete_fixture(fixture_id: int, db: Session = Depends(get_db)):
37     fixture = db.get(Fixture, fixture_id)
38     if fixture is None:
39         raise HTTPException(status_code=404, detail="Fixture not found")
40     db.delete(fixture) # remove the row from MySQL
41     db.commit()
42     return {"message": "Fixture deleted", "id": fixture_id}
```

code > api > main.py > ...

```
1 # HW4 - Community Sports League Fixtures API (Domain 7) for the React client, runs on port 8619
2 from fastapi import FastAPI
3 from fastapi.middleware.cors import CORSMiddleware
4 from .routers.auth import router as auth_router
5 from .routers.fixtures import router as fixtures_router
6 PORT_BASE = 8619
7 app = FastAPI(title="Community Sports League Fixtures - HW4")
8 app.add_middleware(
9     CORSMiddleware,
10     allow_origins=["http://localhost:5173"], # the React dev server
11     allow_credentials=True, # allow the session cookie on cross origin calls
12     allow_methods=["*"],
13     allow_headers=["*"],
14 )
15 app.include_router(auth_router) # POST /login
16 app.include_router(fixtures_router) # CRUD on /fixtures
17 if __name__ == "__main__":
18     import uvicorn
19     uvicorn.run(app, host="127.0.0.1", port=PORT_BASE)
```





```
(data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; SELECT * FROM fixtures;"
Enter password:
+-----+-----+-----+
| id | fixture_title | venue | home_team_id |
+-----+-----+-----+
| 1 | Spartans vs Bulldogs - Matchweek 5 (Rescheduled) | CEFCU Stadium | NULL |
| 2 | Lions vs Tigers - Matchweek 6 | City Arena | NULL |
+-----+-----+-----+
```

Part 3 - N+1 Measurement and Query Tuning

Part 3 measures the N+1 query problem on my fixtures list and fixes it. Each fixture is linked to one home team, so a list that shows fixtures with their team can either run one extra query per fixture (naive) or fetch everything with a single JOIN (fixed). I seeded realistic data, measured both versions at three page sizes, and then added one index and compared the query plans.

1. Seeding 5,000 fixtures and 200 teams

seed_hw04.py uses SEED 3319, so it creates the same data every time: 200 teams and 5,000 fixtures. Each team is the home team for 25 fixtures, so all 200 teams are linked. The script clears both tables and inserts the rows in bulk, using the committed schema 001_schema.sql.

seeded generator and bulk insert

```
def build_rows():
    rng = random.Random(SEED) # seeded generator
    names = [(f"{city} {mascot}", city) for city in CITIES for mascot in MASCOTS] # 20 x 10 = 200 teams
    rng.shuffle(names)
    teams = [{"id": i + 1, "name": n, "city": c} for i, (n, c) in enumerate(names)]
    fixtures = []
    for i in range(N_FIXTURES):
        home = teams[i % N_TEAMS] # every team hosts 25 fixtures
        away = rng.choice([t for t in teams if t["id"] != home["id"]])
        week = rng.randint(1, 38)
        venue = f"{home['city']} {rng.choice(VENUE_TYPES)}" # 100 possible venues
        fixtures.append({"fixture_title": f"{home['name']} vs {away['name']} - Matchweek {week}",
                        "venue": venue, "home_team_id": home["id"]}) # link to the related team
    return teams, fixtures

def main():
    teams, fixtures = build_rows()
    with engine.begin() as conn:
        conn.execute(text("SET FOREIGN_KEY_CHECKS = 0")) # allow clearing tables linked by a foreign key
        conn.execute(text("TRUNCATE TABLE fixtures")) # empty and reset AUTO_INCREMENT to 1
        conn.execute(text("TRUNCATE TABLE teams"))
        conn.execute(text("SET FOREIGN_KEY_CHECKS = 1"))
        conn.execute(insert(Team), teams) # 200 related rows
        conn.execute(insert(Fixture), fixtures) # 5,000 primary rows
    print(f"[{datetime.now().isoformat(timespec='seconds')}] seed={SEED} teams={len(teams)} fixtures={len(fixtures)}")
if __name__ == "__main__":
    main()
```

the related data

```
class Fixture(Base):
    __tablename__ = "fixtures" # primary domain entity
    id = Column(Integer, primary_key=True, autoincrement=True)
    fixture_title = Column(String(200), nullable=False) # primary field
    venue = Column(String(200), nullable=False) # secondary field
    home_team_id = Column(Integer, ForeignKey("teams.id", ondelete="SET NULL"), nullable=True)
    home_team = relationship("Team", back_populates="fixtures")
```

```
class FixtureWithTeam(FixtureOut):
    home_team: TeamOut | None = None # related data for the Part 3 list
```

```
code > api > seed_hw04.py > build_rows
1 # HW4 Part 3: seed 200 teams and 5,000 fixtures with SEED 3319
2 import random
3 from datetime import datetime
4 from sqlalchemy import insert, text
5 from .database import engine
6 from .models import Fixture, Team
7 SEED = 3319 # SID4
8 N_TEAMS = 200
9 N_FIXTURES = 5000
10 CITIES = ["San Jose", "Oakland", "Fremont", "Palo Alto", "Santa Clara", "Sunnyvale", "Milpitas", "Hayward",
11           "Berkeley", "Cupertino", "Campbell", "Mountain View", "Redwood City", "San Mateo", "Daly City",
12           "Walnut Creek", "Pleasanton", "Livermore", "Gilroy", "Morgan Hill"] # 20 cities
13 MASCOTS = ["Spartans", "Bulldogs", "Lions", "Tigers", "Hawks", "Eagles", "Wolves", "Bears", "Falcons", "Sharks"] # 10 names
14 VENUE_TYPES = ["Stadium", "Park", "Arena", "Field", "Sports Complex"]
15 def build_rows():
16     rng = random.Random(SEED) # seeded generator
17     names = [(f"{city} {mascot}", city) for city in CITIES for mascot in MASCOTS] # 20 x 10 = 200 teams
18     rng.shuffle(names)
19     teams = [{"id": i + 1, "name": n, "city": c} for i, (n, c) in enumerate(names)]
20     fixtures = []
21     for i in range(N_FIXTURES):
22         home = teams[i % N_TEAMS] # every team hosts 25 fixtures
23         away = rng.choice([t for t in teams if t["id"] != home["id"]])
24         week = rng.randint(1, 38)
25         venue = f"{home['city']} {rng.choice(VENUE_TYPES)}" # 100 possible venues
26         fixtures.append({"fixture_title": f"{home['name']} vs {away['name']} - Matchweek {week}",
27                         "venue": venue, "home_team_id": home["id"]}) # link to the related team
28     return teams, fixtures
29 def main():
30     teams, fixtures = build_rows()
31     with engine.begin() as conn:
32         conn.execute(text("SET FOREIGN_KEY_CHECKS = 0")) # allow clearing tables linked by a foreign key
33         conn.execute(text("TRUNCATE TABLE fixtures")) # empty and reset AUTO_INCREMENT to 1
34         conn.execute(text("TRUNCATE TABLE teams"))
35         conn.execute(text("SET FOREIGN_KEY_CHECKS = 1"))
36         conn.execute(insert(Team), teams) # 200 related rows
37         conn.execute(insert(Fixture), fixtures) # 5,000 primary rows
38     print(f"[{datetime.now().isoformat(timespec='seconds')}] seed={SEED} teams={len(teams)} fixtures={len(fixtures)}")
39 if __name__ == "__main__":
40     main()
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER
• (data260) villyalsriprajeet@Mac data260-3319 % mkdir -p reports/hw04
• (data260) villyalsriprajeet@Mac data260-3319 % python -m code.api.seed_hw04 2>&1 | tee -a reports/hw04/RUN_LOG.txt
[2026-09-25T16:37:35] seed=3319 teams=200 fixtures=5000
• (data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; SELECT COUNT(*) AS fixtures FROM fixtures; SELECT COUNT(*) AS teams FROM teams; SELECT COUNT(DISTINCT home_team_id) AS teams_linked FROM fixtures; SELECT f.id, f.fixture_title, f.venue, t.name AS home_team FROM fixtures f JOIN teams t ON t.id = f.home_team_id ORDER BY f.id LIMIT 5;"
Enter password:
+-----+
| fixtures |
+-----+
| 5000 |
+-----+
+-----+
| teams |
+-----+
| 200 |
+-----+
+-----+
| teams_linked |
+-----+
| 200 |
+-----+
+-----+
| id | fixture_title | venue | home_team |
+-----+
| 1 | Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 | Pleasanton Sports Complex | Pleasanton Hawks |
| 2 | Santa Clara Hawks vs Fremont Sharks - Matchweek 32 | Santa Clara Field | Santa Clara Hawks |
| 3 | Pleasanton Bears vs Campbell Tigers - Matchweek 17 | Pleasanton Field | Pleasanton Bears |
| 4 | Pleasanton Wolves vs Fremont Eagles - Matchweek 20 | Pleasanton Sports Complex | Pleasanton Wolves |
| 5 | Redwood City Lions vs Morgan Hill Tigers - Matchweek 25 | Redwood City Arena | Redwood City Lions |
+-----+
• (data260) villyalsriprajeet@Mac data260-3319 %
```

2. Naive and fixed list endpoints

Both endpoints return fixtures with their home team, and only logged in users can call them. The naive endpoint runs one query for the page, then one more query per fixture to get its team. The fixed endpoint uses a JOIN, so it gets everything in one query. A small counter tracks every SQL query, and each response reports the total in the X SQL Count header.

```

@router.get("/naive", response_model=list[FixtureWithTeam])
def list_fixtures_naive(limit: int = Query(10, ge=1, le=500), db: Session = Depends(get_db)):
    fixtures = db.query(Fixture).order_by(Fixture.id).limit(limit).all() # 1 query for the page
    for f in fixtures:
        _ = f.home_team # lazy load: 1 extra query per fixture (N+1)
    return fixtures

@router.get("/fixed", response_model=list[FixtureWithTeam])
def list_fixtures_fixed(limit: int = Query(10, ge=1, le=500), db: Session = Depends(get_db)):
    return (db.query(Fixture).options(joinedload(Fixture.home_team)) # teams come in the same query via a JOIN
            .order_by(Fixture.id).limit(limit).all())

```

per-request SQL counter

```

code > api > perf.py > ...
1 # Counts the SQL statements each request sends to MySQL (Part 3)
2 from contextvars import ContextVar
3 from sqlalchemy import event
4 from .database import engine
5 sql_counter = ContextVar("sql_counter", default=None) # {"n": count} for the current request
6 @event.listens_for(engine, "before_cursor_execute")
7 def count_statement(conn, cursor, statement, parameters, context, executemany):
8     box = sql_counter.get()
9     if box is not None:
10         box["n"] += 1 # one statement sent to the database

```

```

@app.middleware("http")
async def add_sql_count(request, call_next):
    box = {"n": 0}
    sql_counter.set(box) # fresh counter for this request
    response = await call_next(request)
    response.headers["X-SQL-Count"] = str(box["n"]) # statements this request ran
    return response

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER
• (data260) villyalsriprajeet@Mac data260-3319 % curl -s -c /tmp/s3319_cookies.txt -X POST http://localhost:8619/login -H "Content-Type: application/json" -d '{"email": "prajeet@s3319.com", "password": "Fixtures123"}'
{"message": "logged in"}
• (data260) villyalsriprajeet@Mac data260-3319 % echo

• (data260) villyalsriprajeet@Mac data260-3319 % for v in naive fixed; do for n in 10 50 200; do printf "%s limit=%s " $v $n; curl -s -b /tmp/s3319_cookies.txt -o /dev/null -D - -http://localhost:8619/fixtures/$v?limit=$n | grep -i x-sql-count; done; done
naive limit=10 x-sql-count: 12
naive limit=50 x-sql-count: 52
naive limit=200 x-sql-count: 202
fixed limit=10 x-sql-count: 2
fixed limit=50 x-sql-count: 2
fixed limit=200 x-sql-count: 2
• (data260) villyalsriprajeet@Mac data260-3319 %

```

The naive count is $N + 2$. The fixed count is always 2.

3. Measurements

measure_hw04.py logs in, sends a few warm up requests, then sends 30 requests to both versions at page sizes 10, 50 and 200. For each request it records the response time and the number of SQL queries. All 180 results are saved in reports/hw04/raw/n1_requests.csv, and the p50/p95/p99 and speed ups are saved in raw/n1_summary.json. The numbers in this report are from the 16:49:24 run.

```

28     for size in PAGE_SIZES:
29         for i in range(1, REQUESTS_PER_CASE + 1):
30             for v in VERSIONS: # alternate naive and fixed so both run under the same conditions
31                 t0 = time.perf_counter()
32                 resp = s.get(f"{BASE}/fixtures/{v}", params={"limit": size})
33                 ms = (time.perf_counter() - t0) * 1000 # end-to-end latency in ms
34                 rows.append({"timestamp": datetime.now().isoformat(timespec="milliseconds"),
35                             "page_size": size, "version": v, "request_no": i,
36                             "status": resp.status_code, "rows_returned": len(resp.json()),
37                             "sql_statements": int(resp.headers["X-SQL-Count"]), "latency_ms": round(ms, 3)})
38     with open(RAW_DIR / "n1_requests.csv", "w", newline="") as f: # all 180 measured requests
39         writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
40         writer.writeheader()
41         writer.writerows(rows)
42     summary = []
43     for size in PAGE_SIZES:
44         for v in VERSIONS:
45             case = [r for r in rows if r["page_size"] == size and r["version"] == v]
46             lat = [r["latency_ms"] for r in case]
47             summary.append({"page_size": size, "version": v, "requests": len(case),
48                             "sql_per_request": statistics.mean(r["sql_statements"] for r in case),
49                             "p50_ms": round(pct(lat, 50), 2), "p95_ms": round(pct(lat, 95), 2),
50                             "p99_ms": round(pct(lat, 99), 2)})
51     for size in PAGE_SIZES: # speed-up = naive p50 / fixed p50
52         naive, fixed = [x for x in summary if x["page_size"] == size]
53         naive["speedup_p50"] = fixed["speedup_p50"] = round(naive["p50_ms"] / fixed["p50_ms"], 2)
54     with open(RAW_DIR / "n1_summary.json", "w") as f:
55         json.dump({"run_at": datetime.now().isoformat(timespec="seconds"), "base_url": BASE,
56                   "requests_total": len(rows), "warmup_per_case": WARMUP, "cases": summary}, f, indent=2)
57     print(f"[{datetime.now().isoformat(timespec='seconds')}] measured {len(rows)} requests")
58     print("| Page size | Version | SQL stmts/req | p50 (ms) | p95 (ms) | p99 (ms) |")
59     print("|---|---|---|---|---|---|")
60     for x in summary:
61         print(f"| {x['page_size']} | {x['version']} | {x['sql_per_request']:g} | {x['p50_ms']} | {x['p95_ms']} | {x['p99_ms']} |")
62     for size in PAGE_SIZES:
63         print(f"page {size}: fixed is {[x for x in summary if x['page_size'] == size][0]['speedup_p50']}x faster (p50)")
64 if __name__ == "__main__":
65     main()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER

```

• (data260) villyalsriprajeet@Mac data260-3319 % python code/measure_hw04.py 2>&1 | tee -a reports/hw04/RUN_LOG.txt
[2026-09-25T16:49:24] measured 180 requests
| Page size | Version | SQL stmts/req | p50 (ms) | p95 (ms) | p99 (ms) |
|---|---|---|---|---|---|
| 10 | naive | 12 | 9.9 | 10.53 | 10.75 |
| 10 | fixed | 2 | 4.56 | 5.17 | 5.49 |
| 50 | naive | 52 | 32.44 | 40.75 | 49.71 |
| 50 | fixed | 2 | 6.24 | 7.15 | 7.49 |
| 200 | naive | 202 | 111.56 | 133.1 | 139.17 |
| 200 | fixed | 2 | 11.25 | 11.98 | 12.22 |
page 10: fixed is 2.17x faster (p50)
page 50: fixed is 5.2x faster (p50)
page 200: fixed is 9.92x faster (p50)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER

```

• (data260) villyalsriprajeet@Mac data260-3319 % ls -l reports/hw04/raw/

total 32
-rw-r--r--  1 villyalsriprajeet  staff   9744 25 Sep 16:49 n1_requests.csv
-rw-r--r--  1 villyalsriprajeet  staff   1382 25 Sep 16:49 n1_summary.json
• (data260) villyalsriprajeet@Mac data260-3319 % wc -l reports/hw04/raw/n1_requests.csv

181 reports/hw04/raw/n1_requests.csv
• (data260) villyalsriprajeet@Mac data260-3319 % head -4 reports/hw04/raw/n1_requests.csv
timestamp,page_size,version,request_no,status,rows_returned,sql_statements,latency_ms
2026-09-25T16:49:18.837,10,naive,1,200,10,12,9.932
2026-09-25T16:49:18.842,10,fixed,1,200,10,2,4.753
2026-09-25T16:49:18.852,10,naive,2,200,10,12,9.914
○ (data260) villyalsriprajeet@Mac data260-3319 % █

```


Results

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	12	9.90	10.53	10.75
10	fixed	2	4.56	5.17	5.49
50	naive	52	32.44	40.75	49.71
50	fixed	2	6.24	7.15	7.49
200	naive	202	111.56	133.10	139.17
200	fixed	2	11.25	11.98	12.22

4. Speed-up and why it changes with page size

Page size	Naive p50 (ms)	Fixed p50 (ms)	Speed up
10	9.90	4.56	2.17x
50	32.44	6.24	5.20x
200	111.56	11.25	9.92x

The fixed version is 2.17x faster at page size 10, 5.20x at 50 and 9.92x at 200. The naive version runs one extra query for every fixture, so the bigger the page, the more queries it runs and the slower it gets. The fixed version always runs just 2 queries, so it stays fast as the page grows. That's why the speed-up gets bigger as the page size increases. The naive version also has more slow outliers, because every extra query is another chance for a delay.

5. Index and EXPLAIN before/after

The list queries were already indexed, so I indexed venue for the query "all fixtures at one venue". The index is saved as the migration code/api/sql/002_add_index.sql.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER
(data260) villyalsripajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; EXPLAIN SELECT id, fixture_title, venue FROM fixtures WHERE venue = 'San Jose Stadium'; SELECT COUNT(*) AS matching_rows FROM fixtures WHERE venue = 'San Jose Stadium';"
Enter password:
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | fixtures | NULL | ALL | NULL | NULL | NULL | NULL | 4890 | 10.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| matching_rows |
+----+-----+
| 35 |
+----+-----+
```

Adding the index

```
code > api > sql > 002_add_index.sql
1 -- HW4 Part 3: index on venue so fixtures at one venue are found without a full table scan
2 USE s3319_rel;
3 CREATE INDEX idx_fixtures_venue ON fixtures (venue);
```

```

(data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p < code/api/sql/002_add_index.sql
Enter password:
(data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; SHOW INDEX FROM fixtures;"
Enter password:
+-----+
| Table           | Non_unique | Key_name      | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Ex |
+-----+
| fixtures | 0 | PRIMARY      | 1 | id          | A         | 4890        | NULL     | NULL   | NULL | BTREE      |         |               | YES     | NU |
| fixtures | 1 | fk_fixtures_team | 1 | home_team_id | A         | 200         | NULL     | NULL   | YES  | BTREE      |         |               | YES     | NU |
| fixtures | 1 | idx_fixtures_venue | 1 | venue       | A         | 100         | NULL     | NULL   | NULL | BTREE      |         |               | YES     | NU |
+-----+

```

After the index

```

(data260) villyalsriprajeet@Mac data260-3319 % mysql -u s3319_app -p -e "USE s3319_rel; EXPLAIN SELECT id, fixture_title, venue FROM fixtures WHERE venue = 'San Jose Stadium';"
Enter password:
+-----+
| id | select_type | table  | partitions | type | possible_keys | key              | key_len | ref | rows | filtered | Extra |
+-----+
| 1  | SIMPLE      | fixtures | NULL       | ref  | idx_fixtures_venue | idx_fixtures_venue | 802     | const | 35 | 100.00 | NULL |
+-----+
(data260) villyalsriprajeet@Mac data260-3319 %

```

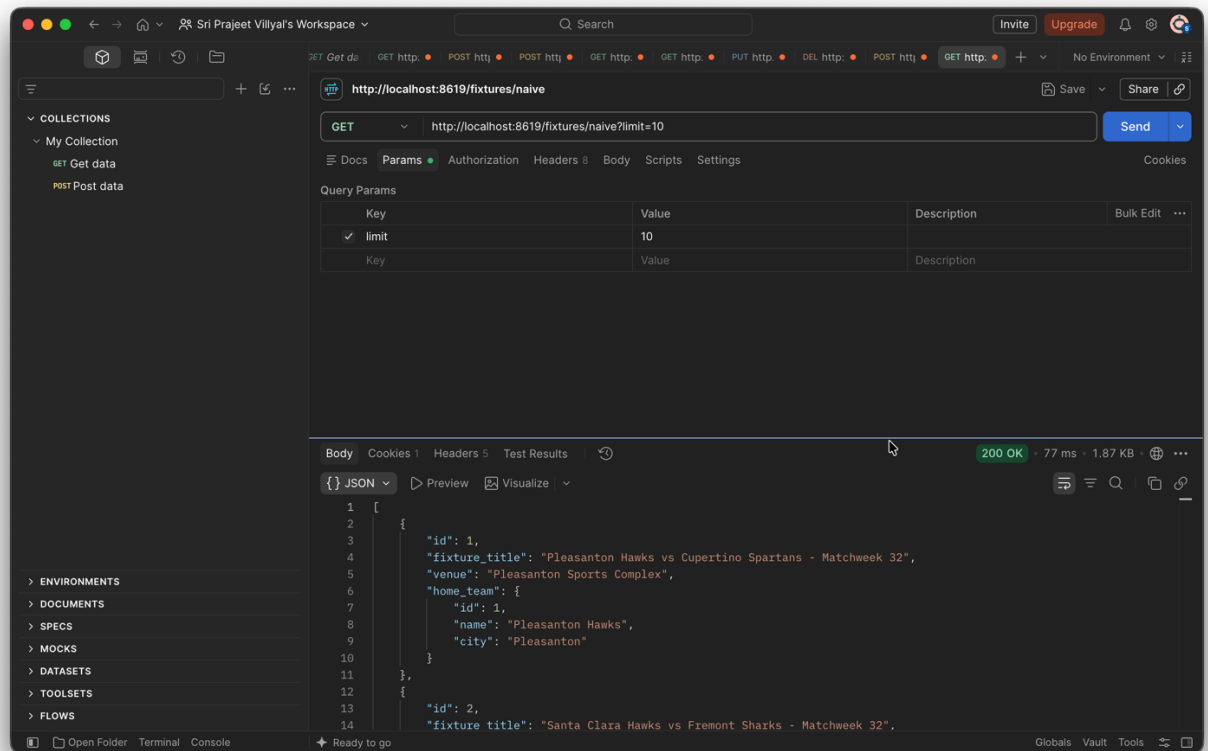
Column	Before	After
type	ALL	ref
key	NULL	idx_fixtures_venue
ref	NULL	const
rows	4890	35
filtered	10.00	100.00
Extra	Using where	NULL

Before the index, MySQL reads the whole table just to find 35 matches. After the index, it goes straight to the 35 rows for 'San Jose Stadium', so the rows it checks drop from about 4,890 to 35.

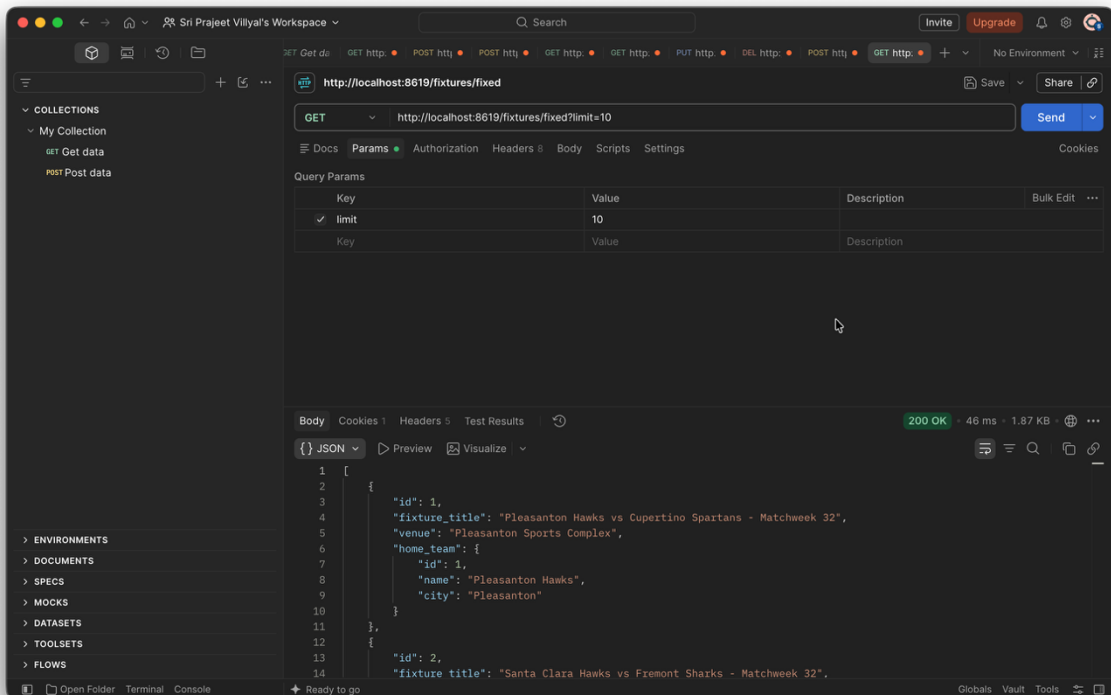
6. Postman

Both versions return the same data, with each fixture and its home team, and the response sizes match at every page size. At page size 200, the headers show 202 SQL queries for naive and only 2 for fixed, for the exact same response. Single Postman timings vary a lot, so the latency numbers in this report come from the 30 request measurement script.

Naive, page size 10



Fixed, page size 10



Naive, page size 50

GET http://localhost:8619/fixtures/naive?limit=50

200 OK 66 ms · 8.65 KB

```
{
  "id": 1,
  "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32",
  "venue": "Pleasanton Sports Complex",
  "home_team": {
    "id": 1,
    "name": "Pleasanton Hawks",
    "city": "Pleasanton"
  },
  "id": 2,
  "fixture_title": "Santa Clara Hawks vs Fremont Sharks - Matchweek 32"
}
```

Fixed, page size 50

GET http://localhost:8619/fixtures/naive?limit=50

200 OK 20 ms · 8.65 KB

```
{
  "id": 1,
  "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32",
  "venue": "Pleasanton Sports Complex",
  "home_team": {
    "id": 1,
    "name": "Pleasanton Hawks",
    "city": "Pleasanton"
  },
  "id": 2,
  "fixture_title": "Santa Clara Hawks vs Fremont Sharks - Matchweek 32"
}
```

Naive, page size 200

GET http://localhost:8619/fixtures/naive?limit=200

200 OK · 184 ms · 34.71 KB

```
{
  "id": 1,
  "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32",
  "venue": "Pleasanton Sports Complex",
  "home_team": {
    "id": 1,
    "name": "Pleasanton Hawks",
    "city": "Pleasanton"
  }
},
{
  "id": 2,
  "fixture_title": "Santa Clara Hawks vs Fremont Sharks - Matchweek 32",
  "venue": "Santa Clara Sports Complex",
  "home_team": {
    "id": 2,
    "name": "Santa Clara Hawks",
    "city": "Santa Clara"
  }
}
```

Fixed, page size 200

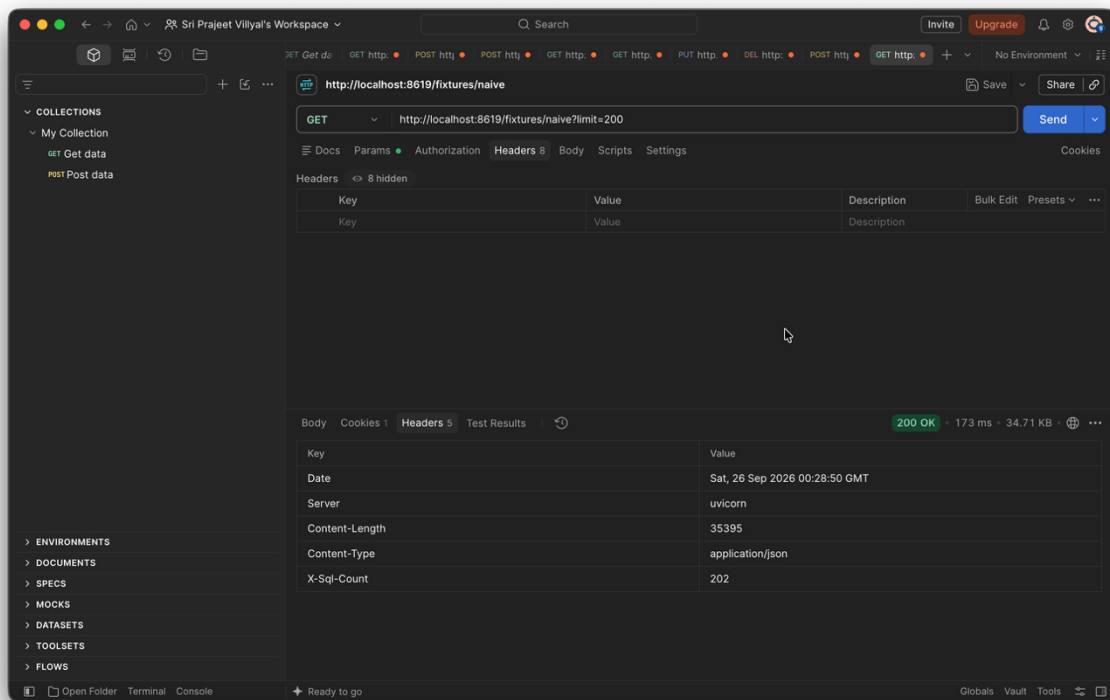
GET http://localhost:8619/fixtures/naive?limit=200

200 OK · 47 ms · 34.71 KB

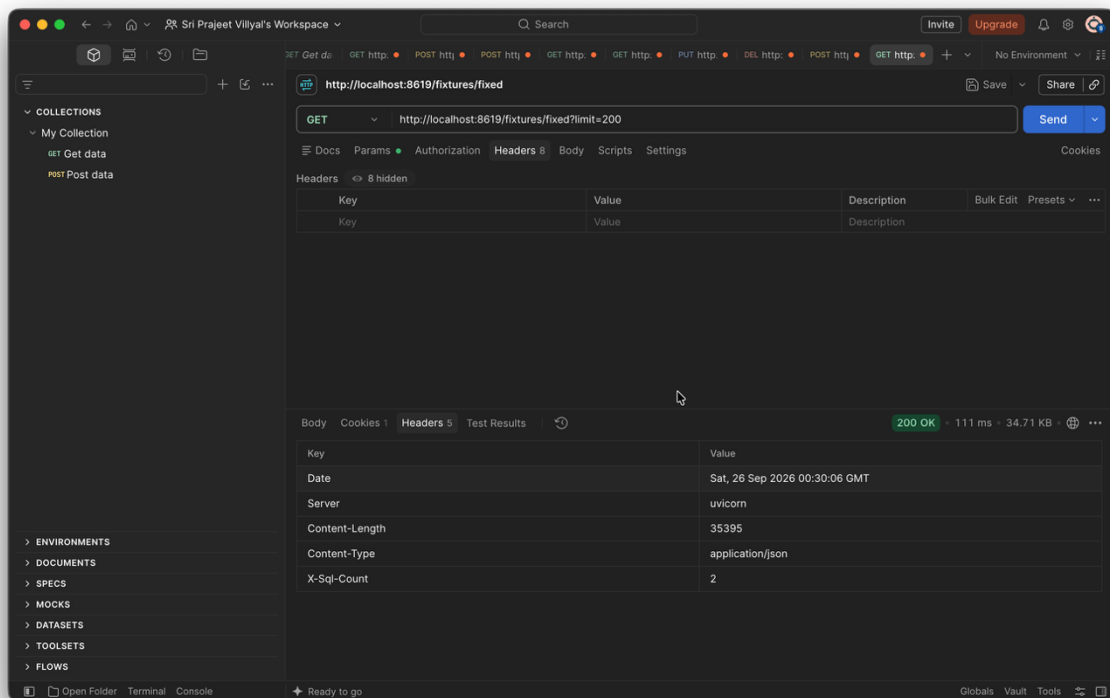
```
{
  "id": 1,
  "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32",
  "venue": "Pleasanton Sports Complex",
  "home_team": {
    "id": 1,
    "name": "Pleasanton Hawks",
    "city": "Pleasanton"
  }
},
{
  "id": 2,
  "fixture_title": "Santa Clara Hawks vs Fremont Sharks - Matchweek 32",
  "venue": "Santa Clara Sports Complex",
  "home_team": {
    "id": 2,
    "name": "Santa Clara Hawks",
    "city": "Santa Clara"
  }
}
```

SQL count in the response headers

Naive 200 headers: X-Sql-Count 202



Fixed 200 headers: X-Sql-Count 2



Part 4 - Grounded RAG Question Answering System

I built a RAG app in code/rag/rag.py that answers questions from my Domain 7 documents. I compared three setups: no RAG, basic RAG and context engineered RAG. Everything runs locally, with qwen2.5:3b (Ollama) for answers and all-MiniLM-L6-v2 for embeddings.

Item	Setting
Corpus	5 Wikipedia season articles from HW3 (data/corpus/, about 310 KB)
Chunking	SentenceSplitter, chunk_size = 500, chunk_overlap = 50 → 248 chunks
Embeddings / store	all-MiniLM-L6-v2 (384 dims), FAISS IndexFlatIP (cosine)
LLM	qwen2.5:3b via Ollama, temperature 0, max 200 answer tokens
Retrieval	top_k = 3 (sweep k = 1, 3, 5)
Outputs	reports/hw04/raw/rag_retrievals.txt, rag_comparison.csv, rag_ksweep.csv, rag_eval.csv, rag_eval_summary.md

```
24 CHUNK_SIZE, CHUNK_OVERLAP = 500, 50 # tokens per chunk and overlap
25 TOP_K = 3
26 MIN_SCORE = 0.35 # config C drops chunks below this cosine score
27 DUP_SIM = 0.95 # config C drops a chunk this similar to one it already kept
28 MAX_CONTEXT_CHARS = 6000 # config C token budget, roughly 1,500 tokens
29 REFUSAL = "I cannot answer this question from the provided documents"
```

1. Corpus and index

load_chunks splits the 5 documents into 500 token chunks with a 50 token overlap, and each chunk keeps its text, source file and chunk_id. build_index embeds the chunks and stores them in a FAISS index. The vectors are normalised, so the scores are cosine similarity.

```
64 def load_chunks():
65     docs = [Document(text=p.read_text(encoding="utf-8"), metadata={"source": p.name})
66              for p in sorted(CORPUS_DIR.glob("*.txt"))]
67     nodes = SentenceSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP).get_nodes_from_documents(docs)
68     chunks, counters = [], {}
69     for n in nodes:
70         src = n.metadata["source"]
71         counters[src] = counters.get(src, 0) + 1
72         chunks.append({"chunk_id": f"{src.removesuffix('.txt')}#{counters[src]}", "source": src,
73                       "text": n.get_content()}) # text, source name and chunk_id for every chunk
74     return docs, chunks
75 def build_index(chunks, model):
76     vecs = model.encode([c["text"] for c in chunks], batch_size=32, normalize_embeddings=True).astype("float32")
77     index = faiss.IndexFlatIP(vecs.shape[1]) # inner product on unit vectors = cosine similarity
78     index.add(vecs)
79     return index, vecs
```

```
• (data260) villalsriprajeet@Villyals-MacBook-Pro data260-3319 % python -u code/rag/rag.py 2>61 | tee -a reports/hw04/RUN_LOG.txt
[2026-09-25T18:58:13] HW4 Part 4 RAG run, model=qwen2.5:3b, embed=sentence-transformers/all-MiniLM-L6-v2
corpus: 5 documents -> 248 chunks (chunk_size=500, overlap=50)
/Users/villyalsriprajeet/miniconda3_x86/envs/data260/lib/python3.12/site-packages/huggingface_hub/file_download.py:1132: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
FAISS index: 248 vectors, dim=384
```

2. Retrieval printed before the LLM

For every question, the top-k chunks are printed with their rank, score, source and chunk_id before any LLM call, so a retrieval problem can be seen separately from a generation problem. The printouts are also saved to raw/rag_retrievals.txt.

retrieval and printout

```
def retrieve(question, k, model, index, chunks, vecs):
    q = model.encode([question], normalize_embeddings=True).astype("float32")
    scores, ids = index.search(q, k) # top-k nearest chunks
    return [{"chunks[i], "rank": r + 1, "score": float(s), "vec": vecs[i]}
            for r, (s, i) in enumerate(zip(scores[0], ids[0]))]
def format_hits(qid, question, k, hits):
    lines = [f"[{qid}] k={k} {question}"]
    for h in hits:
        preview = re.sub(r"\s+", " ", h["text"][:150])
        lines.append(f"#{h['rank']} score={h['score']:.4f} source={h['source']} chunk={h['chunk_id']} | {preview}")
    return "\n".join(lines)
```

3. Three configurations

A sends only the question. B pastes the raw top 3 chunks. C drops chunks below a 0.35 score, drops near duplicates, keeps the context under a size budget, orders the rest by score, labels each with its source, and adds grounding rules: answer only from the context, cite the passage number, and refuse with the exact sentence when the evidence is not there. If no chunk passes the filter, C refuses without calling the model.

context engineering for C

```
def engineer_context(hits):
    kept, dropped = [], []
    for h in sorted(hits, key=lambda x: x["score"], reverse=True): # best evidence first
        if h["score"] < MIN_SCORE:
            dropped.append((h["chunk_id"], "irrelevant"))
        elif any(float(h["vec"] @ k["vec"]) >= DUP_SIM for k in kept):
            dropped.append((h["chunk_id"], "duplicate"))
        elif sum(len(k["text"]) for k in kept) + len(h["text"]) > MAX_CONTEXT_CHARS:
            dropped.append((h["chunk_id"], "over budget"))
        else:
            kept.append(h)
    return kept, dropped
```

grounding rules and the three prompts

```
SYSTEM_C = (
    "You answer questions using ONLY the numbered context passages.\n"
    "Rules:\n"
    "1. Use only facts stated in the context. Do not use outside knowledge.\n"
    "2. Cite the passage number for every fact, like [1] or [2].\n"
    "3. The competition and season in the question must match the context exactly.\n"
    f"4. If the context does not contain the answer, reply exactly: {REFUSAL}\n"
    "5. If the question is ambiguous, say which competition each answer refers to.\n"
    "6. Answer in at most three sentences."
)
```



```
def build_messages(config, question, hits):
    if config == "A": # no RAG baseline
        return [{"role": "user", "content": f"Answer in at most three sentences.\n\nQuestion: {question}"}]
    if config == "B": # basic RAG: raw top-k chunks, no rules
        context = "\n\n".join(h["text"] for h in hits)
        return [{"role": "user", "content": f"Use the context to answer the question.\n\nContext:\n{context}\n\nQuestion: {question}"}]
    context = "\n\n".join(f"[{i}] (source: {h['source']}, chunk {h['chunk_id']})\n{h['text']}"
                          for i, h in enumerate(hits, 1)) # labelled, ordered survivors
    return [{"role": "system", "content": SYSTEM_C},
            {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"}]
```

LLM call and configuration runner

```
def ask_llm(messages, cache):
    key = hashlib.sha256(json.dumps([LLM_MODEL, messages]).encode()).hexdigest()
    if key in cache:
        return cache[key]["answer"], cache[key]["seconds"], True # reuse a saved answer
    payload = {"model": LLM_MODEL, "messages": messages, "stream": False,
              "options": {"temperature": 0, "seed": 3319, "num_ctx": 4096, "num_predict": 200}}
    t0 = time.perf_counter()
    r = requests.post(OLLAMA_URL, json=payload, timeout=900)
    r.raise_for_status()
    answer = r.json()["message"]["content"].strip()
    seconds = round(time.perf_counter() - t0, 1)
    cache[key] = {"answer": answer, "seconds": seconds}
    CACHE_FILE.write_text(json.dumps(cache, indent=1)) # save after every call
    return answer, seconds, False
def run_config(config, q, hits, cache):
    if config == "C":
        kept, dropped = engineer_context(hits) # filter, de duplicate and order
    else:
        kept, dropped = (hits if config == "B" else []), []
    if config == "C" and not kept:
        return {"answer": REFUSAL + ". (No chunk passed the relevance filter.)", "seconds": 0.0,
              "cached": False, "context": [], "dropped": dropped}
    answer, seconds, cached = ask_llm(build_messages(config, q["question"], kept), cache)
    return {"answer": answer, "seconds": seconds, "cached": cached, "context": kept, "dropped": dropped}
```

Six question test set

Q	Type	Question	Expected
Q1	Answer in one chunk	Who won the Golden Boot in the 2023-24 Premier League, and how many goals?	Erling Haaland, 27
Q2	Needs two chunks	Which three clubs were relegated from the 2023-24 Premier League, and which club won the 2023-24 EFL Championship?	Luton, Burnley, Sheffield United; Leicester City
Q3	Similar info across docs	Which club were the defending champions at the start of the 2023-24 La Liga season?	Barcelona

Q4	Ambiguous	Who won the league title?	Depends on the league
Q5	Not in the documents	Who won the 2025 MLS Cup?	Refuse
Q6	Unrelated	What is the capital of Australia?	Refuse

test set

```
QUESTIONS = [
    {"id": "Q1", "type": "answer in one chunk",
     "question": "Who won the Golden Boot in the 2023-24 Premier League, and how many goals did he score?",
     "sources": ["2023_24_premier_league.txt"], "all": ["Haaland", "27"], "any": [], "refuse": False},
    {"id": "Q2", "type": "answer needs two chunks",
     "question": "Which three clubs were relegated from the 2023-24 Premier League, and which club won the 2023-24 EFL Championship?",
     "sources": ["2023_24_premier_league.txt", "2023_24_efl_championship.txt"],
     "all": ["Luton", "Burnley", "Sheffield", "Leicester"], "any": [], "refuse": False},
    {"id": "Q3", "type": "similar information across documents",
     "question": "Which club were the defending champions at the start of the 2023-24 La Liga season?",
     "sources": ["2023_24_la_liga.txt"], "all": ["Barcelona"], "any": [], "refuse": False},
    {"id": "Q4", "type": "ambiguous",
     "question": "Who won the league title?",
     "sources": ["2023_24_premier_league.txt", "2023_24_la_liga.txt", "2023_24_efl_championship.txt",
                  "2024_major_league_soccer_season.txt", "2024_25_uefa_champions_league.txt"],
     "all": [], "any": ["Manchester City", "Real Madrid", "Leicester", "Inter Miami", "LA Galaxy", "Paris Saint-Germain"],
     "refuse": False},
    {"id": "Q5", "type": "answer not in the documents",
     "question": "Who won the 2025 MLS Cup?", "sources": [], "all": [], "any": [], "refuse": True},
    {"id": "Q6", "type": "unrelated",
     "question": "What is the capital of Australia?", "sources": [], "all": [], "any": [], "refuse": True},
]
```

[Q1] k=3 Who won the Golden Boot in the 2023-24 Premier League, and how many goals did he score?
 #1 score=0.5487 source=2023_24_premier_league.txt chunk=2023_24_premier_league#42 | Premier League. Retrieved 20 May 2026. ^ "Postecoglou ends wait for Australia with Barclays Manager award". Premier League. 15 September 2023. Archive
 #2 score=0.5294 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#22 | Includes 3 goals in The Championship play-offs. Hat-tricks [edit] Main article: List of EFL Championship hat-tricks | Player | For | Against | Result
 #3 score=0.5218 source=2023_24_premier_league.txt chunk=2023_24_premier_league#20 | article: List of Premier League hat-tricks | Player | For | Against | Result | Date | Son Heung-min | Tottenham Hotspur | Burnley | 5-2 (A) | 2 September
 (A) 67.9s -> The player who won the Golden Boot in the 2023-24 Premier League was Mohamed Salah, scoring 22 goals.
 (B) 305.4s -> Based on the information provided in the context, there is no specific mention of the Golden Boot winner or the number of goals scored in the 2023-24 Premier League season. The context discusses various awards and achievements for players, but does not provide details about the Golden Boot winner or
 (C) 233.0s -> I cannot answer this question from the provided documents

[Q2] k=3 Which three clubs were relegated from the 2023-24 Premier League, and which club won the 2023-24 EFL Championship?
 #1 score=0.6603 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#2 | Team changes [edit] The following teams have changed division since the 2022-23 season: < The template below (Col-begin) is being considered for deletion
 #2 score=0.6471 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#36 | Cup | Prospects | Promotion to Premier League Relegation to EFL League One | v t e Second level English football league seasons | Football League Season
 #3 score=0.6205 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#1 | Football league season EFL Championship | Season | 2023-24 | Dates | 11 August 2023 - 4 May 2024 | Champions | Leicester City 2nd Championship title 8t
 (A) 59.2s -> In the 2023-24 Premier League, three clubs were relegated: Sheffield United, Norwich City, and Brighton & Hove Albion. The club that won the 2023-24 EFL Championship was Brighton & Hove Albion.
 (B) 296.7s -> Based on the information provided in the context:
 - The three clubs that were relegated from the 2023-24 Premier League (which is referred to as the EFL Championship in this context) were Birmingham City, Huddersfield Town, and Rotherham United.
 - Leicester City won the 2023-24 EFL Championship.
 (C) 277.6s -> The three clubs relegated from the 2023-24 Premier League were Birmingham City, Huddersfield Town, and Rotherham United [1] and [4]. Leicester City won the 2023-24 EFL Championship [4].

[Q3] k=3 Which club were the defending champions at the start of the 2023-24 La Liga season?
 #1 score=0.6625 source=2023_24_la_liga.txt chunk=2023_24_la_liga#2 | It commenced on 11 August 2023 and ended on 26 May 2024. Barcelona were the defending champions, having won their 27th title the previous season. On 4
 #2 score=0.6298 source=2023_24_la_liga.txt chunk=2023_24_la_liga#23 | 22 February 2023. ^ "Spanish LALIGA Performance Stats, 2023-24 Season". ESPN. Retrieved 26 May 2024. ^ "EA SPORTS & LaLiga Announce Expansive New Part
 #3 score=0.6252 source=2023_24_la_liga.txt chunk=2023_24_la_liga#1 | 93rd season of La Liga Football league season La Liga | Season | 2023-24 | Dates | 11 August 2023 - 26 May 2024 | Champions | Real Madrid 36th title |
 (A) 42.6s -> The defending champions at the start of the 2023-24 La Liga season were Real Madrid.
 (B) 233.9s -> According to the context provided, Barcelona were the defending champions at the start of the 2023-24 La Liga season. They had won their 27th title the previous season.
 (C) 284.4s -> Barcelona were the defending champions at the start of the 2023-24 La Liga season, as stated in [1].

[Q4] k=3 Who won the league title?
 #1 score=0.4571 source=2023_24_premier_league.txt chunk=2023_24_premier_league#5 | The defeat left the Blades 10 points adrift of safety with three matches left to play. On 4 May, they also became the first Premier League side to con
 #2 score=0.4528 source=2023_24_premier_league.txt chunk=2023_24_premier_league#17 | 1-2 | 2-3 | 2-1 | 1-1 | 2-4 | 1-1 | - | 1-2 | 1-2 | 1-0 | 1-1 | 1-3 | 0-1 | 1-2 | 1-1 | Manchester City | 0-0 | 4-1 | 6-1 | 1-0 | 2-1 | 3-1 | 1-1 | 2-
 #3 score=0.4464 source=2024_25_uefa_champions_league.txt chunk=2024_25_uefa_champions_league#17 | Teams ranked from 25th to 36th were eliminated from all competitions, with no access to the 2024-25 UEFA Europa League. | Pos | Team v t e | Pld | W |
 (A) 37.6s -> I'm sorry, but I don't have enough information to answer who won the league title. Could you please specify which league you are referring to?
 (B) 240.4s -> Based on the information provided in the context, Manchester City won the league title. They secured the title with 91 points by beating West Ham United 3-1 at home on the final day of the season.
 (C) 224.0s -> Manchester City won the league title with 91 points.

[Q5] k=3 Who won the 2025 MLS Cup?
 #1 score=0.6502 source=2024_major_league_soccer_season.txt chunk=2024_major_league_soccer_season#19 | Open Cup winners; (V) Canadian Championship winners Notes: ^ As 2024 MLS Supporters' Shield champions ^ As winners of Leagues Cup ^ As runners-up of L
 #2 score=0.5672 source=2024_major_league_soccer_season.txt chunk=2024_major_league_soccer_season#47 | ^ "MLS unveils 2024 Regular Season schedule". MLSsoccer.com. December 20, 2023. Archived from the original on December 20, 2023. Retrieved December 20
 #3 score=0.5572 source=2024_major_league_soccer_season.txt chunk=2024_major_league_soccer_season#68 | Major League Soccer. November 7, 2024 . Retrieved November 7, 2024. ^ Sigal, Jonathan (November 4, 2024). "LA Galaxy's Gabriel Pec named 2024 MLS Newco
 (A) 48.1s -> As of my last update in 2023, the MLS Cup winner for 2025 has not been determined yet, as the tournament takes place annually. To find the correct winner, you should check the latest news or official announcements from Major League Soccer.
 (B) 307.8s -> Based on the information provided in the context, the 2024 MLS Cup was won by LA Galaxy. The context mentions "LA Galaxy lift MLS Cup 2024 over New York Red Bulls" on December 7, 2024. However, the question asks about the 2025 MLS Cup, which is not mentioned in the given context. Therefore, I cannot
 (C) 213.4s -> I cannot answer this question from the provided documents

[Q6] k=3 What is the capital of Australia?
 #1 score=0.1962 source=2024_25_uefa_champions_league.txt chunk=2024_25_uefa_champions_league#3 | Teams | Notes | 1 | England | 109.570 | 4 | 2 | Spain | 92.998 | 3 | Germany | 82.481 | +1 (EPS) | 4 | Italy | 81.926 | +1 (EPS) | 5 | France |
 #2 score=0.1681 source=2024_major_league_soccer_season.txt chunk=2024_major_league_soccer_season#23 | Please help update this article to reflect recent events or newly available information. (February 2025) As of May 15, 2024 | Rank | Team | GP | Cumul
 #3 score=0.1503 source=2024_25_uefa_champions_league.txt chunk=2024_25_uefa_champions_league#4 | Teams | Notes | 20 | Greece | 25.225 | 1 | 21 | Israel | 25.000 | 22 | Cyprus | 24.475 | 23 | Sweden | 23.750 | 24 | Poland | 20.750 | 25
 (A) 12.2s -> The capital of Australia is Canberra.
 (B) 234.1s -> The capital of Australia is Canberra.
 (C) 0.0s -> I cannot answer this question from the provided documents. (No chunk passed the relevance filter.)
 dropped: [('2024_25_uefa_champions_league#3', 'irrelevant'), ('2024_major_league_soccer_season#23', 'irrelevant'), ('2024_25_uefa_champions_league#4', 'irrelevant')]

Q	Right chunks?	A: No RAG	B: Basic RAG	C: Context RAG
Q1	No	Wrong: "Salah, 22"	Says not in context	Exact refusal
Q2	No (EFL only)	Wrong: Norwich, Brighton	Relegated wrong; Leicester right	Relegated wrong; Leicester right; cites missing [4]
Q3	Yes	Wrong: Real Madrid	Barcelona	Barcelona [1]
Q4	PL only	Asks which league	Manchester City	Manchester City (no citation)
Q5	n/a	No answer, no refusal sentence	Explains 2025 not covered	Exact refusal
Q6	n/a	Canberra (outside knowledge)	Canberra	Refused by filter

Context size sweep (k = 1, 3, 5)

Q2 was run through config C at k = 1, 3 and 5.

```
SWEEP_QID, SWEEP_KS = "Q2", [1, 3, 5] # context size sweep
```

```
for k in SWEEP_KS:
    hits = retrieve(sq["question"], k, model, index, chunks, vecs)
    block = format_hits(sq["id"], sq["question"], k, hits)
    print(block)
    printouts.append(block)
    res = run_config("C", sq, hits, cache)
    ev = evaluate(sq, "C", res, hits)
    print(f" (C, k={k}) {res['seconds']}s -> {res['answer'][:300]}")
```

```
=== top_k sweep on Q2 (config C) ===
[Q2] k=1 Which three clubs were relegated from the 2023-24 Premier League, and which club won the 2023-24 EFL Championship?
#1 score=0.6603 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#2 | Team changes [edit] The following teams have changed divi
sion since the 2022-23 season: < The template below (Col-begin) is being considered for delet
(C, k=1) 48.0s -> Leicester City, Leeds United, and Southampton were relegated from the 2023-24 Premier League. [1]
[Q2] k=3 Which three clubs were relegated from the 2023-24 Premier League, and which club won the 2023-24 EFL Championship?
#1 score=0.6603 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#2 | Team changes [edit] The following teams have changed divi
sion since the 2022-23 season: < The template below (Col-begin) is being considered for delet
#2 score=0.6471 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#36 | Cup | Prospects | Promotion to Premier League Relegation
to EFL League One | v t e Second level English football league seasons | Football League Seco
#3 score=0.6205 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#1 | Football league season EFL Championship | Season | 2023-2
4 | Dates | 4 August 2023 - 4 May 2024 | Champions | Leicester City 2nd Championship title 8t
(C, k=3) 277.6s -> The three clubs relegated from the 2023-24 Premier League were Birmingham City, Huddersfield Town, and Rotherham United [1] a
nd [4]. Leicester City won the 2023-24 EFL Championship [4].
[Q2] k=5 Which three clubs were relegated from the 2023-24 Premier League, and which club won the 2023-24 EFL Championship?
#1 score=0.6603 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#2 | Team changes [edit] The following teams have changed divi
sion since the 2022-23 season: < The template below (Col-begin) is being considered for delet
#2 score=0.6471 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#36 | Cup | Prospects | Promotion to Premier League Relegation
to EFL League One | v t e Second level English football league seasons | Football League Seco
#3 score=0.6205 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#1 | Football league season EFL Championship | Season | 2023-2
4 | Dates | 4 August 2023 - 4 May 2024 | Champions | Leicester City 2nd Championship title 8t
#4 score=0.5995 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#9 | April 2024 | 24th | Steve Evans | 17 April 2024 League ta
ble [edit] | Pos | Team | Pld | W | D | L | GF | GA | GD | Pts | Promotion, qualification or
#5 score=0.5936 source=2023_24_efl_championship.txt chunk=2023_24_efl_championship#34 | ^ "Sky Bet Championship Manager and Player of the Month f
or November". EFL. 8 December 2023. ^ "Sky Bet Championship Manager and Player of the Month f
(C, k=5) 348.0s -> The three clubs relegated from the 2023-24 EFL Championship were Birmingham City, Huddersfield Town, and Rotherham United [7]
, [8], [9]. Leicester City won the 2023-24 EFL Championship [6].
```

k	Chunks used	Result	Time
1	EFL #2	Wrong season (2022-23 relegations), champion missing	48 s
3	EFL #2, #36, #1	Leicester correct, relegated clubs wrong, invalid citation [4]	278 s
5	+ EFL #9, #34	Leicester correct, relegated clubs wrong, invalid citations [6]-[9]	348 s

A higher k did not help. The Premier League relegation chunk was never retrieved at any k. At k = 5 the model got a league table and a references list, which made it slower and led it to cite passages that didn't exist. k = 3 was the best balance.

6. Evaluation

For each question and setup, the script checks whether the retrieval was right, whether the answer was right, whether the answer was grounded, and whether it refused when it should. It saves the results in raw/rag_eval.csv.

```
def evaluate(q, config, res, hits):
    ans = res["answer"]
    refused = "cannot answer this question from the provided documents" in ans.lower()
    ctx = res["context"]
    ctx_text = " ".join(c["text"] for c in ctx)
    retrieved_text = " ".join(h["text"] for h in hits)
    if q["refuse"]:
        retrieval_ok = "n/a"
    elif q["all"]:
        retrieval_ok = has(retrieved_text, q["all"]) # the retrieved chunks hold every fact needed
    else:
        retrieval_ok = any(w.lower() in retrieved_text.lower() for w in q["any"])
    if q["refuse"]:
        answer_ok = refused
    elif q["all"]:
        answer_ok = has(ans, q["all"]) and not refused
    else:
        answer_ok = any(w.lower() in ans.lower() for w in q["any"]) and not refused
    cited = [int(n) for n in re.findall(r"\\(\\d+)\\", ans)]
    if config == "A":
        grounded = "n/a" # no context to be grounded in
    elif refused:
        grounded = True
    else:
        facts = [w for w in q["all"] + q["any"] if w.lower() in ans.lower()]
        source_text = " ".join(ctx[n - 1]["text"] for n in cited if 1 <= n <= len(ctx)) if config == "C" else ctx_text
        grounded = bool(facts) and all(f.lower() in source_text.lower() for f in facts)
    if config == "C":
        format_ok = refused if q["refuse"] else (bool(cited) and all(1 <= n <= len(ctx) for n in cited))
    else:
        format_ok = "n/a"
    return {"correct_retrieval": retrieval_ok, "correct_answer": answer_ok, "grounded": grounded,
            "refused_when_needed": refused if q["refuse"] else "n/a", "format_ok": format_ok}
```

Config	Accuracy	Faithfulness	Format compliance	Robustness (Q5/Q6)
A: No RAG	0/6	n/a	n/a	0/2
B: Basic RAG	2/6	3/6	n/a	0/2
C: Context-engineered RAG	4/6	5/6	3/6	2/2

```
=== evaluation summary ===
| Config | Accuracy (correct answers) | Faithfulness (grounded) | Format compliance | Robustness (Q5/Q6 refused) |
|---|---|---|---|
| A: No RAG | 0/6 | n/a | n/a | 0/2 |
| B: Basic RAG | 2/6 | 3/6 | n/a | 0/2 |
| C: Context-engineered RAG | 4/6 | 5/6 | 3/6 | 2/2 |
```

7. Analysis

Retrieval decided most of the results. Only Q3 retrieved the right chunk at rank 1 (the La Liga chunk naming Barcelona as defending champions, score 0.66), and both RAG setups answered it correctly while No RAG said Real Madrid. Q1 failed at retrieval: the top chunks were a references list and hat trick tables (0.52–0.55), and the Haaland 27 goal Golden Boot sentence was never retrieved, likely because all-MiniLM-L6-v2 only reads the first approx 256 tokens of each 500 token chunk, so a smaller chunk size should help. B said the answer was not in the context and C refused, so neither guessed, while No RAG invented "Salah, 22". Q2 needed chunks from two documents, but all three came from the EFL Championship document because it repeats "relegated", "Premier League" and "2023–24", so both RAG setups named the wrong relegated clubs and only Leicester was correct. The top_k sweep showed that more context is not better context: k = 1 gave last season's relegations, k = 3 found Leicester, and k = 5 only added a league table and a references list, ran 25% slower and cited passages that did not exist, so k = 3 was best. Of the context changes, the relevance filter and grounding rules helped most. For Q6 the filter removed every chunk (0.15–0.20), so C refused without calling the model, while A and B answered "Canberra" from outside knowledge. For Q5 the 2024 MLS chunks looked relevant (0.55–0.65), but the grounding rule made C refuse with the exact sentence; B only explained that 2025 was not covered and A gave a general disclaimer, so only C met the refusal requirement, although no configuration invented a 2025 winner. Source labels and ordering let C cite passages, but the 3B model did not always follow the format: it cited a missing passage on Q2 and gave no citation on Q4, so format compliance was only 3/6. On the ambiguous Q4, No-RAG asked which league was meant, while both RAG setups answered "Manchester City" because retrieval returned only Premier League chunks. Overall, retrieval quality set the ceiling, context quality decided what evidence reached the model, and the grounding prompt decided whether it refused or guessed.

AI Use

1. What I used an AI assistant for and what I did myself:

I used an AI assistant to explain each step and help write the auth API, the React pages, the N+1 measurement script, the index migration and the RAG script. I ran every command myself on my Mac, checked the results in the terminal, Postman, MySQL and the browser, took all the screenshots, and wrote up the results.

2. One AI output that was wrong/unsuitable:

The first RAG script crashed with a "segmentation fault" on my Mac as soon as it built the FAISS index. Earlier, the suggested pip install cryptography also failed because it tried to build the newest version from source on my x86 conda environment.

3. How I detected the problem / verified the result:

The crash appeared right after the embeddings loaded and before any question ran, so the problem was loading FAISS and PyTorch together, not my questions. For cryptography, the error said there was no wheel and OpenSSL was missing. After each fix I re-ran the step and checked the output: all 6 questions and the k-sweep finished and were saved to reports/hw04/raw, and import pymysql connected to MySQL.

4. What I changed and why it works now:

I set KMP_DUPLICATE_LIB_OK=TRUE and OMP_NUM_THREADS=1 and imported sentence-transformers before faiss, so the two libraries share one OpenMP runtime and no longer crash. For cryptography I used pip install --only-binary=:all: cryptography, which installed a ready-made wheel (48.0.1) instead of building from source.

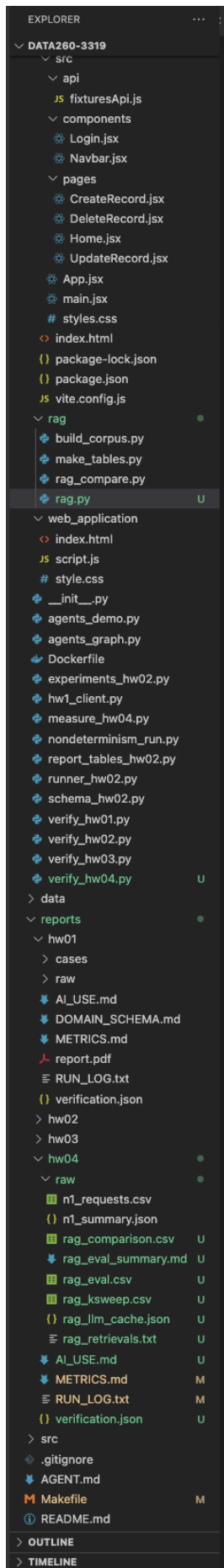
Verification

```
(data260) villyalsriprajet@Villyals-MacBook-Pro data260-3319 % make verify-hw04
python code/verify_hw04.py
PASS - file exists: code/api/main.py
PASS - file exists: code/api/database.py
PASS - file exists: code/api/models.py
PASS - file exists: code/api/security.py
PASS - file exists: code/api/routers/auth.py
PASS - file exists: code/api/routers/fixtures.py
PASS - file exists: code/api/perf.py
PASS - file exists: code/api/seed_hw04.py
PASS - file exists: code/api/sql/001_schema.sql
PASS - file exists: code/api/sql/002_add_index.sql
PASS - file exists: code/measure_hw04.py
PASS - file exists: code/rag/rag.py
PASS - file exists: code/frontend/package.json
PASS - file exists: code/frontend/src/App.jsx
PASS - file exists: code/frontend/src/components/Login.jsx
PASS - file exists: code/frontend/src/pages/Home.jsx
PASS - file exists: code/frontend/src/pages/CreateRecord.jsx
PASS - file exists: code/frontend/src/pages/UpdateRecord.jsx
PASS - file exists: code/frontend/src/pages/DeleteRecord.jsx
PASS - file exists: reports/hw04/RUN_LOG.txt
PASS - file exists: reports/hw04/METRICS.md
PASS - file exists: reports/hw04/AI_USE.md
PASS - raw N+1 data has 180 requests (180 rows)
PASS - raw N+1 data covers 3 page sizes x 2 versions (('10', 'fixed'), ('10', 'naive'), ('200', 'fixed'), ('200', 'naive'), ('50', 'fixed'), ('50', 'naive'))
PASS - raw data: naive runs more SQL than fixed at page 200 (naive {202}, fixed {2})
PASS - RAG output exists: raw/rag_retrievals.txt
PASS - RAG output exists: raw/rag_comparison.csv
PASS - RAG output exists: raw/rag_ksweep.csv
PASS - RAG output exists: raw/rag_eval.csv
PASS - RAG output exists: raw/rag_eval_summary.md
PASS - RAG eval covers 6 questions x 3 configs (18 rows)
PASS - RAG config C refused Q5 and Q6 (['True', 'True'])
PASS - RAG top_k sweep ran k = 1, 3, 5 (['1', '3', '5'])
PASS - database connects via db_session_base26 engine (mysql+pymysql://s3319_app:***@localhost:3306/s3319_rel)
PASS - fixtures table has at least 5,000 rows (5000)
PASS - teams table has 200 rows, all linked (200 teams, 200 linked)
PASS - users table has a login account (1)
PASS - sessions table exists
PASS - index idx_fixtures_venue exists
PASS - backend responds on port 8619
PASS - list endpoint without login returns 401 (401)
PASS - login returns 200 (200)
PASS - session cookie is HttpOnly
PASS - naive list returns 10 fixtures with home_team (200)
PASS - fixed list returns 10 fixtures with home_team (200)
PASS - naive and fixed return the same data
PASS - naive runs more SQL than fixed (naive 12, fixed 2)
PASS - GET by id works (id 1923 from VERIFY_SEED) (200)
PASS - POST creates a record with an id (201)
PASS - PUT updates the record (200)
PASS - DELETE removes the temp record (delete 200, then get 404)
PASS - Ollama has qwen2.5:3b

52/52 checks passed -> reports/hw04/verification.json
```

```
M Makefile
1   verify-hw02:
2   |   python code/verify_hw02.py
3   verify-hw03:
4   |   python code/verify_hw03.py
5   verify-hw04:
6   |   python code/verify_hw04.py
```

Project folder structure



Conclusion:

For HW4 I extended my existing repo with four parts. In Part 1 I built a React client for my fixtures domain, with a Login page, protected Home, Create, Update and Delete pages, React Router, useState and useEffect hooks, and props for each CRUD action, and it talks to the backend on port 8619 using an HttpOnly session cookie. In Part 2 I moved my fixtures data from memory to MySQL (s3319_rel) with SQLAlchemy and the db_session_based connection. I replaced the HW3 signed cookie with server side sessions stored in a sessions table, and tested all five CRUD endpoints in Postman. In Part 3 I seeded 5,000 fixtures and 200 teams with SEED 3319 and measured a naive and a fixed list endpoint at page sizes 10, 50 and 200. The naive version ran 202 queries for 200 fixtures while the fixed version always ran 2, making it up to 9.92x faster. Adding a venue index cut the rows MySQL checked from about 4,890 to 35. In Part 4 I built a local RAG app with qwen2.5:3b, all-MiniLM-L6-v2 and FAISS over my five season documents, and compared No RAG, Basic RAG and context engineered RAG on six questions plus a k sweep. The context engineered version was the only one that refused Q5 and Q6 correctly, and retrieval quality turned out to matter the most for good answers.